

Integridad y Autenticidad: Funciones Resumen y Certificados

Práctica 2.2 - Presencial

1. Objetivo

El objetivo de esta práctica será el de familiarizarse con los conceptos de **integridad y autenticidad** a través del uso de **funciones resumen y de certificados**. En esta parte se abordarán los conceptos relacionados con las Funciones Resumen (Hash), vinculándola con la práctica anterior a través de la firma de mensajes a encriptar/desencriptar.

2. Firma Mensajes

Además de conseguir confidencialidad (ya vista en la práctica anterior), a través de la firma de mensajes podemos conseguir **autenticación**, entendiendo que el usuario que envía ese mensaje es el que debe ser y no otro. Para ello, lo que se realizará, será la firma del mensaje a encriptar (no lo encriptaremos en esta práctica ya que lo hicimos en la anterior) a través de funciones hash.

Para ello, copia la solución en la que utilizaste cifrado asimétrico a través de la librería *cryptography* para hacer uso de la misma y proceder con la firma del mensaje creado. En este caso llama a la nueva solución ***firma_mensaje.py*** para diferenciarla de la anterior.

Verifica que la solución es funcional y que se crea el mensaje inicial (a cifrar) de forma correcta. Este será el mensaje a firmar.

Procede a realizar la **firma del mensaje con clave privada**. Esto permitirá a cualquier usuario que tenga la clave pública verificar que el mensaje fue creado por alguien que tiene la clave privada. Las firmas en RSA requieren de una función hash específica (función para transformar cualquier bloque arbitrario de datos en una nueva serie de caracteres de longitud fija) y un relleno (*padding*) para poder ser utilizado. También se utilizará un *salt* que consistirá en un conjunto de bits aleatorios que se usa como entrada en una función.

Para hacer uso de las funciones resumen (*hash*) y del relleno (*padding*), añade los siguientes módulos a la solución:

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
```

Ahora atiende a la variable que tenía el texto a cifrar, ya que será el mensaje que firmemos. Por simplicidad, se recomienda crear una nueva variable (*mensaje_firmar*) para evitar confusión. A esta variable será a la que le pases un *string* que determine el texto que se firmará, por ejemplo: "Mensaje a firmar".

Crea una variable llamada *firma* que llame al método ***sign()*** de la clave privada, que será con la que firmemos el mensaje. A este método, habrá que pasarle 3 parámetros, el mensaje a firmar, el relleno a utilizar (*padding*) y la función hash.

El mensaje se pasará sin definir propiedades ya que es un *string* creado en la variable *mensaje_firmar*. No olvides definirlo como cadena de bytes (añadiendo “b” al mensaje) para que pueda ser tratado correctamente desde los métodos de cifrado/descifrado.

Ejemplo: `mensaje_firmar = b"Mensaje a firmar"`

Utilizaremos el método PSS (*Probabilistic Signature Scheme*) para el *padding* ya que es más complejo y seguro que PKCS1. A este método hay que pasarle 2 parámetros: *mgf* que será un objeto de función para la generación de máscara (únicamente soporta MGF1) y *salt_length* que determinará el tamaño del *salt* a utilizar (se suele emplear el tamaño máximo *PSS.MAX_LENGTH*).

Como ayuda, se ofrece la definición del *padding* a utilizar:

```
padding.PSS(  
    mgf=padding.MGF1(hashes.SHA256()),  
    salt_length=padding.PSS.MAX_LENGTH  
)
```

Como podemos ver, se crea el relleno mediante PSS atendiendo a los dos parámetros mencionados anteriormente. Usaremos MGF1 para definir *mgf* pasándole una función hash, en este caso que use SHA (*Secure Hash Algorithm*) de 256 bits empleado en seguridad criptográfica.

El último parámetro vendrá definido como la función hash a utilizar que deberá ser la misma que en la definición del relleno. Por tanto estableceremos como método SHA256() quedando de la siguiente forma:

```
hashes.SHA256()
```

Con esto conseguimos tener en la variable *firma* la firma generada con la clave privada que teníamos disponible anteriormente y el mensaje (*string*) firmado.

Para verificarlo, **imprime por pantalla la firma**.

Para verificar que la firma cambia, crea otra nueva variable llamada *mensaje_firma2* con un nuevo mensaje (por ejemplo, “Mensaje a Firmar 2”) y ahora crea una nueva firma (variable *firma2*) para este mensaje. Imprime por pantalla esta nueva firma para ver si se corresponde a la anterior o por el contrario ha cambiado.

Con esto conseguiremos tener mensajes firmados que pueden ser cifrados (atendiendo a la anterior práctica) para poder enviarlos a un destinatario. A través de esto, conseguimos autenticación de datos y a través del cifrado confidencialidad.

3. Funciones Resumen

Como ya sabemos, las funciones resumen, también llamadas funciones hash o simplemente hash, son algoritmos que consiguen crear una salida alfanumérica de longitud normalmente fija que representa un resumen de la información, a partir de una entrada de datos.

Principalmente, esto se traduce en un proceso criptográfico generado por un algoritmo, pero se diferencia con el resto de métodos criptográficos en que este no puede descifrarse, es decir, con este método no es posible devolver el valor original del dato de entrada.

Por tanto, tendremos algo parecido a esto:



A la vista de la figura superior, no podemos a partir del “Valor Hash” deducir la entrada de datos.

En esta parte de la práctica, realizaremos varios cifrados de datos usando funciones hash que devolverán un valor hash determinado, así como diferentes pruebas para verificar el funcionamiento de este tipo de algoritmo.

3.1 Módulo hashlib

La librería estándar de Python ya nos propone el uso del módulo **hashlib**, cuya documentación está disponible en: (<https://docs.python.org/3/library/hashlib.html>). Este módulo consigue implementar una interfaz común para el uso de múltiples algoritmos seguros de hash. Entre ellos se incluyen los siguientes: FIPS, SHA1, SHA224, SHA256 (ya usado), SHA384 y SHA512 (definidos en FIPS 180-2) y también el algoritmo MD5 de RSA (definido en Internet RFC 1321).

Pese a que los términos más usados hoy en día son “Hash Seguro” y “Resumen del mensaje” (son lo mismo), los algoritmos más antiguos se solían llamar algoritmos de “Digestión de Mensajes” (o “Funciones de Digestión”), de ahí usar el método `digest()` en algunos casos.

En este módulo existe un método constructor para cada tipo de hash. Todos los métodos devolverá un objeto hash con la misma interfaz independientemente del método que se use. En cualquier instante se puede pedir el resumen de la concatenación de los datos introducidos hasta ese momento atendiendo al método mencionado anteriormente `digest()` y al método `hexdigest()` que devolverá realizará la misma función pero en hexadecimal.

Para probar todo lo explicado hasta el momento, vamos a utilizar el hash **SHA256** para cifrar un mensaje e imprimirlo por pantalla.

En primer lugar, crea una nueva solución llamada, **prueba_hash.py** e importa el módulo **hashlib** que hemos explicado anteriormente haciendo uso de la sentencia:

```
import hashlib
```

A continuación crea una nueva variable llamada *mensaje* que contendrá un *string*, por ejemplo, “Mensaje a hashear”. Recuerda declararlo con la “b” inicial para tratarla como cadena de bytes.

Crea otra nueva variable llamada *mensaje_hash* a la que, haciendo uso del objeto **hashlib** se llame al método `sha256()` al que se le pasará el mensaje inicial.

A continuación, imprime dicha variable pasándole los métodos descritos anteriormente (`digests()` y `hexdigests()`) para verificar cómo se puede ver la información del hash que obtenemos en ambos casos:

```
PS C:\Users\Admin\Desktop\PracticaB2.2\Funciones_Hash> & C:/Users/Admin/AppData/Local/Microsoft/Windows/PowerShell/PowerShell.exe -c python C:/Users/Admin/Desktop/PracticaB2.2/Funciones_Hash/prueba_hash.py
Hash: b'i\xac\xd9B\x02\x15\x01\x94\xd6\x92q\x01\n\x143\x08%\xf0pD4\xd75G\x841\xacA$JG3'
Hash Hexadecimal: 69acd94202150194d69271010a14330825f0704434d753478431ac41244a4733
PS C:\Users\Admin\Desktop\PracticaB2.2\Funciones_Hash>
```

Ejecuta el programa varias veces (con 2 o 3 es suficiente) para verificar que el valor del hash es siempre el mismo ya que el mensaje no cambia.

Ahora, realizaremos algunas **pruebas de concepto** sobre el cálculo de valores hash, modificando algunos parámetros y verificando los hash encontrados.

1. Como primera prueba, **cambia el mensaje inicial** de la variable *mensaje* para que ahora tenga el valor "Texto a hashear" (cambiamos la palabra *Mensaje* por *Texto*) y vuelve a ejecutar el programa. (Puedes utilizar una nueva variable para ir apilando las diferentes pruebas). ¿Qué valor ofrecen ahora los métodos `digets()` y `hexdigets()`? ¿Varía con respecto a la ejecución inicial de la solución?. Al cambiar el mensaje, el resultado será totalmente diferente. Con esto podemos evitar que, si un atacante "captura" nuestro mensaje (por ejemplo en un ataque MitM (*Man-In-The-Middle*), pueda modificar el valor inicial del mismo ya que no se podría obtener a partir del hash y si se modifica el emisor sabría que se ha modificado.
2. Prueba a **modificar el algoritmo SHA** por otro de la misma familia, por ejemplo por **SHA512** y vuelve a ejecutarlo. ¿Qué obtenemos ahora? ¿Cambia el Hash? ¿Es más pequeño o más grande?
3. Además de la familia de algoritmos SHA, el módulo *hashlib* puede usar otros como ya explicamos antes. Prueba ahora a modificar el **algoritmo por MD5** y compara el resultado que obtienes con el caso anterior. ¿Qué se obtiene? ¿Cuál consideras que, a priori, es más seguro? ¿Por qué?
4. La función `new()` devuelve un nuevo objeto de la clase hash que implementa la función hash que se le especifique. En este caso, el primer parámetro deberá ser una cadena con el nombre de la función hash que se quiera utilizar ("sha1", "md5", "sha256", etc.) y el segundo parámetro cualquier tipo de cadena que queramos cifrar. Añade el uso de esta función a las pruebas realizadas anteriormente. Para ello, crea una variable a la que asignes una llamada a la función `new` pasándole el algoritmo "sha256" y el mensaje *b"texto"*. Imprime esta variable utilizando el método `digets()` y `hexdigets()`. Cambia el algoritmo a otro de los conocidos y comprueba de nuevo la salida.
5. La función `update()` actualizará el objeto hash añadiendo nueva información (normalmente otra cadena). Pese a que se realice más de una llamada al método, será lo equivalente a realizar una única llamada. Utiliza este método para realizar el hash de un mensaje que actualizarás posteriormente. Para ello, crea una variable (*mensaje*) y asigna directamente la llamada al objeto *hashlib* usando el método `sha256()`. Posteriormente utiliza la función `update()` 3 veces (ya no hace falta usar el objeto *hashlib*) para actualizar el valor de la variable *mensaje* creada anteriormente. Puedes introducir el texto que quieras, por ejemplo, "*mensaje*" en la primera actualización, "*super*" en la segunda y "*secreto*" en la tercera. Posteriormente, muestra por pantalla el mensaje.

Hacer esto, equivaldría a realizar lo siguiente:

```
mensaje = hashlib.sha256(b"mensaje" + b"super" + b"secreto")
```

6. Por último, **verifica la longitud de todos los hash generados** en las pruebas realizadas con anterioridad. Para ello, utiliza el atributo `digest_size` en las variables de mensajes encriptados mediante hash que has utilizado anteriormente.

3.2 Funciones resumen desde librería cryptography

En esta ocasión, atenderemos a las opciones que nos ofrece la librería *cryptography* para el uso de funciones resumen (hash). Veremos casos muy similares a los expuestos anteriormente con el módulo *hashlib*. Para acceder a toda la documentación relativa a este tipo de funciones resumen, puedes acceder a: <https://cryptography.io/en/latest/hazmat/primitives/cryptographic-hashes/>.

En primer lugar, y por no acumular muchas líneas de código y mezclar conceptos, crea una nueva solución llamada *hash_cryptography.py* para resolver esta parte de la práctica.

Para hacer uso de resúmenes de mensaje (Hashing), tendremos que importar la función *default_backend()* y el módulo *hashes* para poder hacer uso de funciones resumen en este sentido. Para ello, incorpora lo siguiente:

```
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import hashes
```

Una importados, ya podremos utilizar unas operaciones parecidas a lo que utilizamos en el apartado anterior para llevar a cabo tareas de hash.

Para comprobar el funcionamiento básico de este tipo de operaciones utilizando la librería *cryptography* en primer lugar, crea un objeto llamado *digest* que será a la que le pasemos el hash a utilizar. Asigna a ese objeto una llamada al método *Hash()* haciendo uso del módulo *hashes*, al que le tendremos que pasar 2 parámetros: el algoritmo de hash a utilizar y el *backend*.

Pasa como algoritmo *SHA256()* y por como el segundo parámetro pasa el *backend* por defecto (*default_backend()*). Actualiza a través de *update()* el contenido de *digest*, puedes añadir un par de mensajes como “Mensaje” + “Secreto”. Recuerda añadir la “b” inicial a los mensajes.

Por último, utiliza el método *finalize()* sobre el objeto *digest*. Este método finaliza el trabajo en el contexto actual y devuelve el mensaje *digest* como bytes.

Imprime por pantalla las características del objeto *digest* creado. En este caso, deberás imprimir la cadena de bytes que devuelve el objeto, el propio objeto para verificar lo que devuelve, y el algoritmo que está usando dicho objeto. Deberás obtener algo parecido a esto:

```
PS C:\Users\Admin\Desktop\PracticaB2.2\Funciones_Hash> & C:/Users/Admin/AppData/Local/Microsoft
b'#Y\xd1\xdb\xdd\xa7\x10I\xc4\x03\x8a[]W-\\xb2\x17\xf4\x07\xd4\x05v@M\x13\xdd\xe8!\xf4\n\xf0'
<cryptography.hazmat.bindings._rust.openssl.hashes.Hash object at 0x000001B791DADB70>
<cryptography.hazmat.primitives.hashes.SHA256 object at 0x000001B791D8A060>
PS C:\Users\Admin\Desktop\PracticaB2.2\Funciones_Hash>
```

Ahora como **prueba final**, cambia el mensaje anterior (puedes usar el mismo programa y cambiar la cadenas a “Secret” + “Message” para verificar si el hash calculado desde la librería *cryptography* también cambia al igual que pasaba con la librería *hashlib*. ¿Cambia? ¿Se modifica el valor del objeto creado cuando mostramos el algoritmo del mismo en cada ejecución?. Intenta razonar el porqué de estos posibles cambios/no cambios.

Realiza alguna **prueba extra** sobre el programa realizado haciendo uso de la librería *cryptography* para verificar su funcionamiento. Por ejemplo, **cambia el algoritmo** elegido para ver cómo se comporta el uso de funciones hash en este tipo de aplicaciones.

Integridad y Autenticidad: Funciones Resumen y Certificados

Práctica 2.2 - Online

1. Objetivo

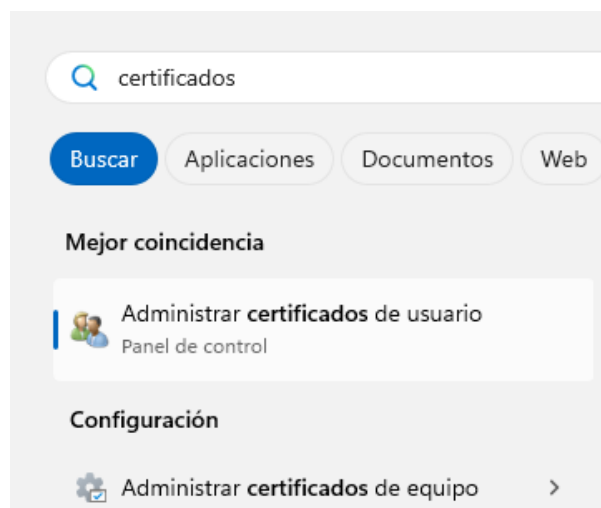
El objetivo de esta parte de la práctica es la de tomar conciencia y práctica sobre el **manejo de certificados en entornos Windows**. Para ello, se hará uso de los certificados de la FNMT que tenéis disponibles, así como certificados generados por nosotros mismos.

Deberás usar la máquina anfitriona y/o la máquina virtual para llevar a cabo esta parte de la práctica. De esta forma, tendrás los certificados instalados y configurados para poder usarlos posteriormente.

2. Certificados de Windows

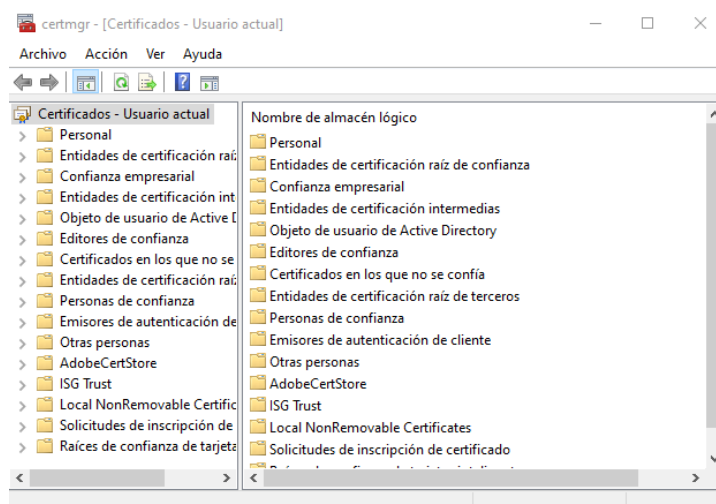
Windows cuenta con un almacén de certificados que serán los que se puedan manejar desde diferentes ámbitos pudiendo obtener firmas digitales que garantizarán la autenticidad e integridad de los datos.

En este caso, Windows almacena los certificados en dos almacenes independientes. Por un lado los Certificados de Usuario y por otro lado los Certificados de Equipo (se necesita ser administrador para poder administrarlos). Para abrir cualquier de ellos puedes hacerlo desde *Panel de Control* → *Cuentas de Usuario* → *Administrar Certificados de Usuario* o *Panel de Control* → *Herramientas Administrativas* → *Administrar Certificados de Equipo*, para cada uno de los casos, pero para abreviar la búsqueda, pulsa sobre el botón de inicio de Windows y escribe la palabra “*Certificados*”.

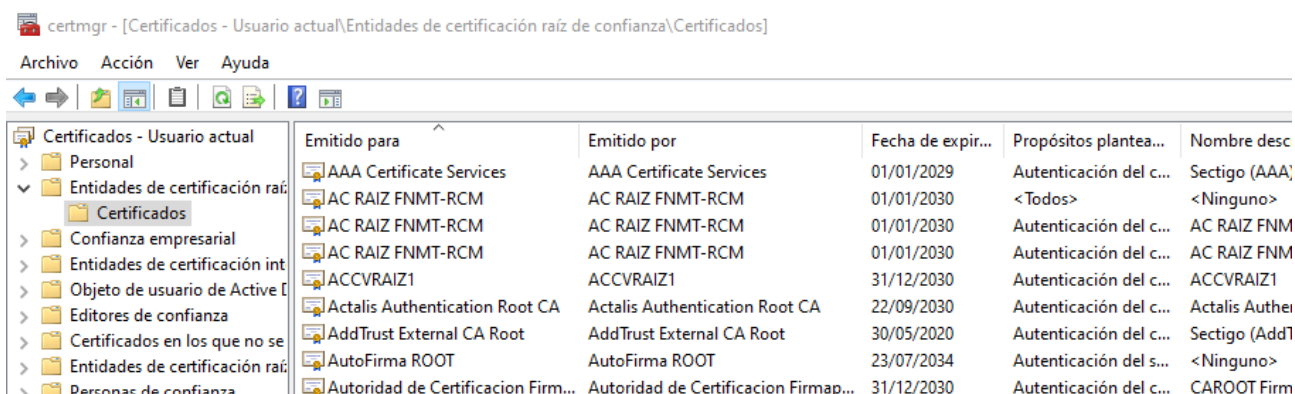


Al realizar esa búsqueda vemos como directamente nos aparecen las dos herramientas que determinarán, por un lado, los certificados de usuario y por otro, los certificados de equipo.

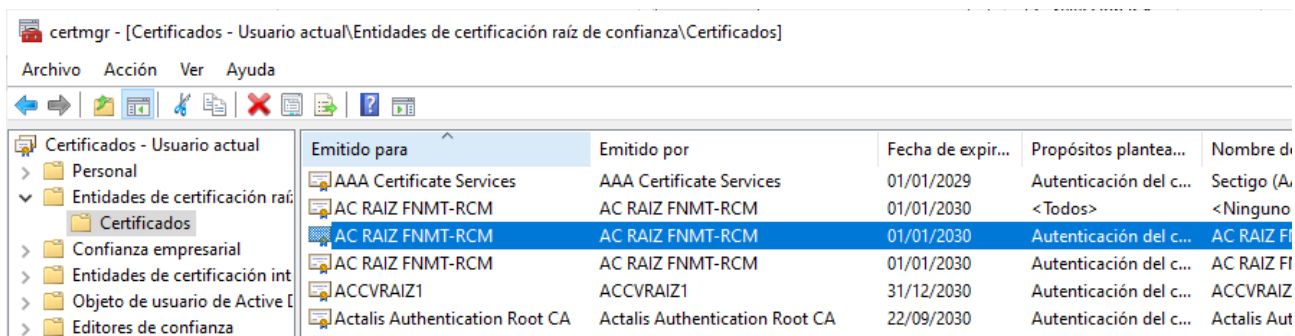
En esta ocasión nos centraremos únicamente en los **certificados de usuario** por lo que, abre la herramienta para poder gestionarlos.



Si desplegamos el árbol de la consola (ventana de la izquierda) y entramos en *Entidades de certificación raíz de confianza > Certificados*, vemos algunos de los certificados que tenemos:



Observar que el cuarto botón por la izquierda está pulsado porque se está mostrando el árbol de la consola. Cuando se selecciona un certificado en el panel derecho, aparecen botones a la derecha que permiten cortar, copiar, eliminar, ver propiedades, y exportar la lista. Estas tareas también se pueden realizar desplegando el menú "Acción". **Selecciona un certificado** para poder comprobar que aparecen esas opciones en el menú.



PROPIEDADES DE UN CERTIFICADO:

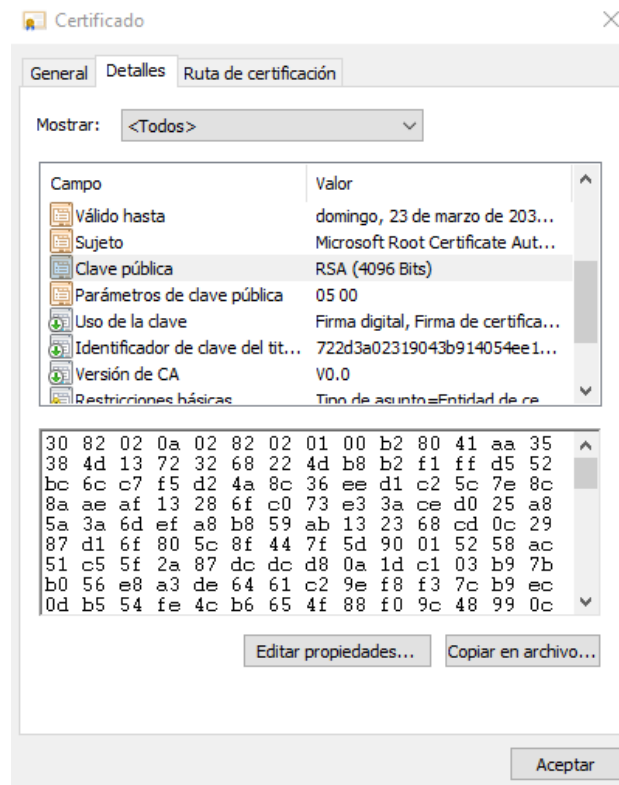
En el menú Acción elegir la opción Propiedades, o pulsar el botón derecho del ratón sobre el certificado seleccionado, o pulsar el botón Propiedades y aparece esta ventana cuando está seleccionado un certificado. Observar que este certificado está habilitado para todos los propósitos. Las otras pestañas tienen sus opciones vacías.



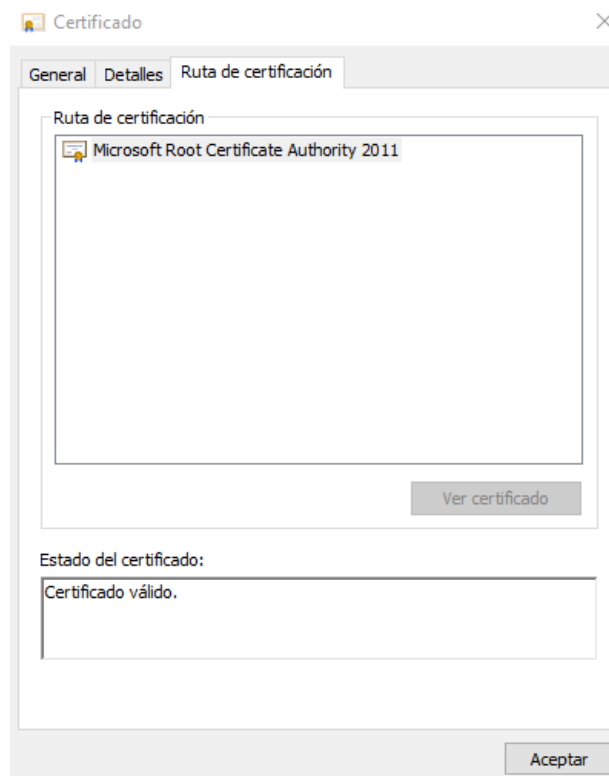
Si elegimos la opción de *Abrir* el certificado aparece la ventana *Certificado* que tiene tres pestañas. En la pestaña *General* se muestra la información general del certificado. Los propósitos del certificado, emitido para, emitido por y su periodo de validez. Observar que como éste es un certificado raíz, entonces "*Emitido para == Emitido por*".



En la pestaña *Detalles* se muestra el contenido del certificado. Observar cómo al seleccionar un campo del certificado se muestra el contenido del campo seleccionado en la ventana inferior. En la figura siguiente se ha seleccionado el campo Clave pública.



En la pestaña *Ruta de certificación* se muestra la ruta. Observar que en el caso de un certificado raíz no hay ruta alguna. También nos muestra el estado del certificado (Certificado válido).



Abre otros certificados y comprueba sus propiedades para ver y valorar las posibles diferencias. Se propone que se abra alguno que no sea Certificado Raíz. Por ejemplo, puedes **incorporar y abrir el certificado personal de la FNMT** para comprobar los datos del certificado, quien lo emite, clave pública asociada, ruta de certificación, etc.

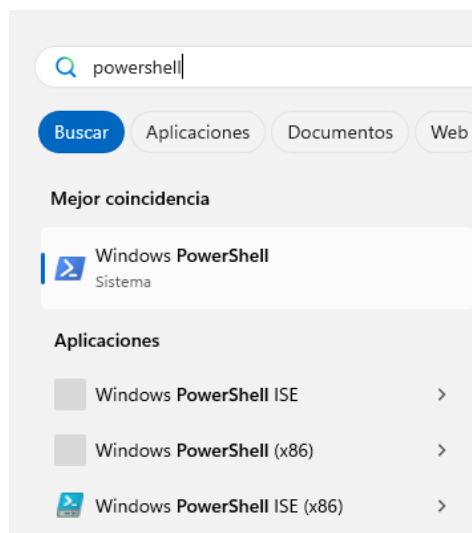
3. Creación de Certificados

Hay múltiples formas de crear los certificados necesarios.

- 1) Utilizando una autoridad de certificación pública a la que se puede acceder vía web. Como ejemplo se puede visitar <https://letsencrypt.org/es/>.
- 2) Instalando una autoridad certificadora propia que emita certificados. Un ejemplo es el servicio "Certificate Server" de Windows Server, que permite implementar un Infraestructura de Clave Pública (PKI) corporativa. También se puede emular el funcionamiento de una autoridad certificadora con una herramienta como <https://www.openssl.org/> que es de uso común.
- 3) Usando un sencillo programa que permita crear certificados. Los programas makecert.exe y pvk2pfx.exe están disponibles en un subdirectorío del Sistema Windows o de Visual Studio. En el Campus Virtual se deja una copia de ambos programas (en su versión de 32 y 64 bits) para llevar a cabo el Anexo A.
- 4) A través de *cmdlet* en PowerShell usando el comando *New-SelfSignedCertificate*. **En esta práctica se generarán certificados de este modo en la Máquina Virtual de Prácticas.**

A partir de los sistemas operativos Windows 10 y Windows Server 2016 el entorno de PowerShell proporciona el comando (*cmdlet*) *New-SelfSignedCertificate* que permite crear certificados para comprobar el funcionamiento de sistemas y aplicaciones. Estos certificados solo se deben utilizar para hacer pruebas, no para un uso normal ya que son autofirmados y por tanto no certificados por una entidad certificadora.

Abrir una consola de PowerShell en un sistema operativo Windows 11. Para ello pulsa el botón inicio y busca "PowerShell".



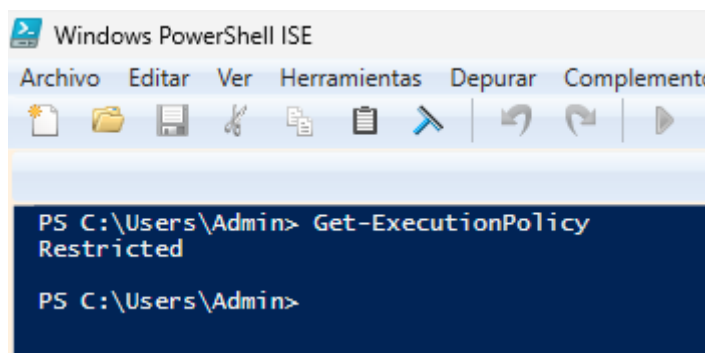
En un SO de 64 bits aparece la aplicación "Windows PowerShell" de 64 bits y también la aplicación "Windows PowerShell (x86)" de 32 bits. También aparece "Windows PowerShell (ISE)" que es el "Integrated Scripting Environment", un entorno de desarrollo de scripts integrado y su equivalente de 32 bits.

La web con toda la documentación de PowerShell (PS) es:

<https://docs.microsoft.com/en-us/powershell/>

Abre el PowerShell ISE, ya que proporciona ayuda para desarrollar los scripts.

Generalmente, la **ejecución de scripts** en un sistema estará **restringida**. Usar el comando *Get-ExecutionPolicy* para comprobarlo:



Para activar la ejecución usar el comando: *Set-ExecutionPolicy -Scope CurrentUser Unrestricted*. Si aparece un cuadro de dialogo pulsa a "Si a todo". Si se cierra la sesión de PS y luego se abre una nueva, en la nueva sesión la política de ejecución permanece *Unrestricted*, verifícalo para no tener problemas futuros.

```
PS C:\Users\Admin> Set-ExecutionPolicy -Scope CurrentUser Unrestricted
PS C:\Users\Admin> Get-ExecutionPolicy
Unrestricted
PS C:\Users\Admin> |
```

PARA GENERAR UN CERTIFICADO RAÍZ

Se puede utilizar el siguiente script:

```
$cert = New-SelfSignedCertificate -Type Custom `
-Subject "CN=zpac.as" `
-KeyAlgorithm RSA -KeyLength 2048 -KeySpec Signature -KeyExportPolicy Exportable `
-KeyUsageProperty All -KeyUsage None `
-Provider "Microsoft Enhanced RSA and AES Cryptographic Provider" `
-NotBefore (Get-Date) `
-NotAfter (Get-Date).AddYears(10) `
-HashAlgorithm sha256 `
-TextExtension @"(2.5.29.19={critical}{text}ca=1)" `
-CertStoreLocation "Cert:\CurrentUser\My"
```

Este script simplemente utiliza el *cmdlet New-SelfSignedCertificate* para generar un nuevo certificado autofirmado y su clave privada asociada. El certificado se carga en el almacén de certificados del usuario y su clave asociada en el almacén de claves del usuario. Además, ambos elementos (certificado y su clave) se asignan a la variable *\$cert*, para su posterior uso en la sesión de PowerShell.

Observar que se utiliza el carácter ` (acento invertido) como indicador de continuación de línea. Se puede insertar en el texto con Alt+96 (poner 96 en el teclado numérico). Mira bien que todos los elementos se corresponden a los que hay indicados en el guión.

La imagen siguiente muestra la edición del script en el ISE y su ejecución:

```

1 $cert = New-SelfSignedCertificate -Type Custom
2 -Subject "CN=zpac.as"
3 -KeyAlgorithm RSA -KeyLength 2048 -KeySpec Signature -KeyExportPolicy Exportable
4 -KeyUsage CertSign, CRLSign, DigitalSignature, KeyEncipherment, DataEncipherment
5 -NotBefore (Get-Date)
6 -NotAfter (Get-Date).AddYears(10)
7 -HashAlgorithm sha256
8 -TextExtension @"("2.5.29.19={critical}{text}ca=1")`
9 -CertStoreLocation "Cert:\CurrentUser\My"

PS C:\Windows\system32> C:\Users\Admin\Desktop\PracticaB2.2\Certificados\CertificadoRaiz.ps1
PS C:\Windows\system32>
  
```

Esta figura muestra la edición del script **CertificadoRaiz.ps1** (crea certificado powershell Autoridad Certificadora). Guarda el script en el fichero. Observar los parámetros del comando:

-Type: Especifica el tipo de certificado creado. Aquí se utiliza el tipo Custom. Otros tipos son CodeSigningCert, DocumentEncryptionCert y SSLServerAuthentication (defecto).

-KeyAlgorithm: Especifica el algoritmo para el que se crean las claves asimétricas asociadas al certificado. Los valores posibles son RSA y ECDSA.

-KeyLength: Especifica la longitud en bits de la clave que es asociada con el nuevo certificado. No existe un valor por defecto.

-KeySpec: Especifica si la clave privada asociada con el nuevo certificado se puede usar para firmar, cifrar o ambas cosas. Los valores aceptables son KeyExchange, Signature y None (defecto). El valor None indica que se usa el valor por defecto que utiliza el proveedor de servicios criptográficos.

-KeyExportPolicy: Especifica la política que gobierna la exportación de la clave privada asociada con el certificado. Los valores aceptables son: Exportable, ExportableEncrypted (defecto) y NonExportable.

-keyUsageProperty: Especifica los usos de la clave para la propiedad "Usos de clave" de la clave privada. Los valores aceptables para este parámetro son: All, Decrypt, KeyAgreement, None (defecto) y Sign. El valor None indica que el comando usa el valor por defecto que utilice el proveedor de servicios de claves.

-KeyUsage: Especifica los usos de clave establecidos en la extensión de uso de clave del certificado. Los valores aceptables para este parámetro son: CertSign, CRLSign, DataEncipherment, DecipherOnly, DigitalSignature, EncipherOnly, KeyAgreement, KeyEncipherment, None (defecto) y NonRepudiation. El valor predeterminado, None, indica que este cmdlet no incluye la extensión KeyUsage en el nuevo certificado. El uso de la clave se restringe a los valores especificados en este parámetro. Por ello es mejor no restringir el uso indicando None.

-Provider: Especifica el nombre del proveedor de servicios criptográficos (CSP) o del proveedor de almacenamiento de claves (KSP). Consultar la ayuda para determinar los proveedores disponibles. Si no se indica nada se determina un proveedor en función del parámetro -KeySpec. Por defecto: "Microsoft Base Cryptographic Provider v1.0". Es **esencial** usar "Microsoft Enhanced RSA and AES Cryptographic Provider" para evitar limitaciones en el uso de las claves privadas.

-NotBefore: Indica la fecha de inicio del período de validez del certificado.

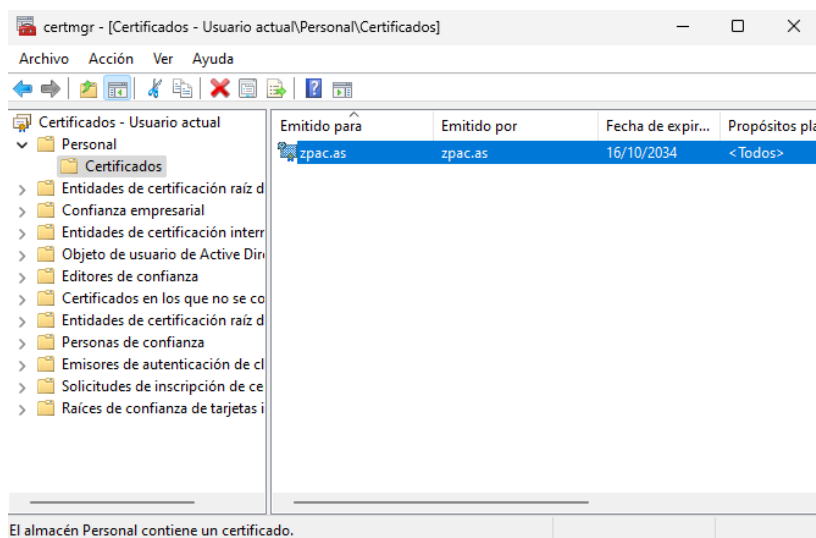
-NotAfter: Indica la fecha de finalización del período de validez del certificado.

-HashAlgorithm: Especifica el nombre del algoritmo de hash usado en la firma del nuevo certificado. El algoritmo por defecto depende del proveedor que almacena la clave privada usada para firmar el nuevo certificado.

-TextExtension: En este caso se utiliza para indicar que el certificado es de una Autoridad Certificadora. Es necesario para que el navegador Firefox permita cargarlo en el almacén de raíces de confianza.

-CertStoreLocation: Especifica el almacén en que se almacena el nuevo certificado. Solo se puede especificar dos almacenes de certificados: Cert:\CurrentUser\My o Cert:\LocalMachine\My. NO se pueden usar otros almacenes de certificados.

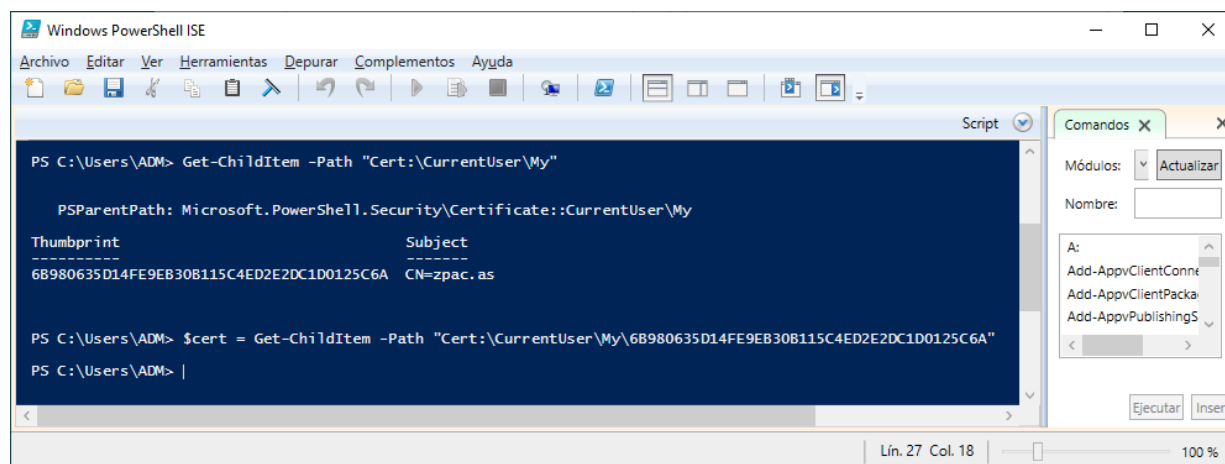
Utiliza la herramienta certmgr.msc para comprobar que el certificado emitido para zpAC está en el almacén de certificados "Personal".



Este almacén NO es el apropiado para contener el certificado de una autoridad certificadora. Pero recordar que este certificado y su clave asociada se usarán para crear otros certificados, y no como raíz de confianza en el computador.

PARA GENERAR UN CERTIFICADO DE SERVIDOR

Si se cerró la sesión de PowerShell en la que se generó el certificado de la Autoridad Certificadora y se cargó el certificado en la variable \$cert, hay que ejecutar estos 2 comandos:

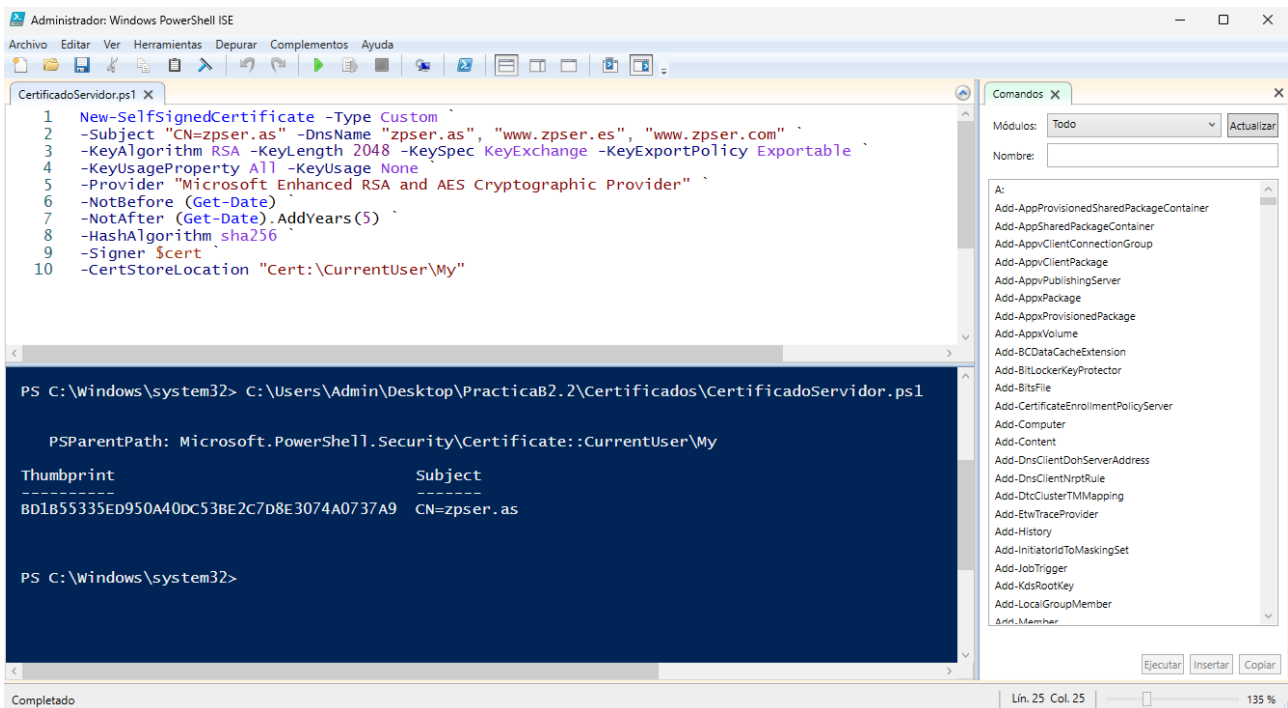


El primer comando permite ver la huella digital (*Thumbprint*) de los certificados. Hay que copiar la huella del certificado de **zpac.as** en el segundo comando para cargar la información del certificado en la variable **\$cert**.

Ahora se puede generar un certificado para un servidor con el siguiente script:

```
New-SelfSignedCertificate -Type Custom `
-Subject "CN=zpser.as" -DnsName "zpser.as", "www.zpser.es", "www.zpser.com" `
-KeyAlgorithm RSA -KeyLength 2048 -KeySpec KeyExchange -KeyExportPolicy Exportable `
-KeyUsageProperty All -KeyUsage None `
-Provider "Microsoft Enhanced RSA and AES Cryptographic Provider" `
-NotBefore (Get-Date) `
-NotAfter (Get-Date).AddYears(5) `
-HashAlgorithm sha256 `
-Signer $cert `
-CertStoreLocation "Cert:\CurrentUser\My"
```

Tras ejecutar este script, el ISE indica la generación del nuevo certificado:



```
Administrador: Windows PowerShell ISE
CertificadoServidor.ps1
1 New-SelfSignedCertificate -Type Custom
2 -Subject "CN=zpser.as" -DnsName "zpser.as", "www.zpser.es", "www.zpser.com"
3 -KeyAlgorithm RSA -KeyLength 2048 -KeySpec KeyExchange -KeyExportPolicy Exportable
4 -KeyUsageProperty All -KeyUsage None
5 -Provider "Microsoft Enhanced RSA and AES Cryptographic Provider"
6 -NotBefore (Get-Date)
7 -NotAfter (Get-Date).AddYears(5)
8 -HashAlgorithm sha256
9 -Signer $cert
10 -CertStoreLocation "Cert:\CurrentUser\My"

PS C:\Windows\system32> C:\Users\Admin\Desktop\PracticaB2.2\Certificados\CertificadoServidor.ps1

PSParentPath: Microsoft.PowerShell.Security\Certificate::CurrentUser\My

Thumbprint      Subject
-----
BD1B55335ED950A40DC53BE2C7D8E3074A0737A9 CN=zpser.as

PS C:\Windows\system32>
```

Observar los nuevos parámetros de este script.

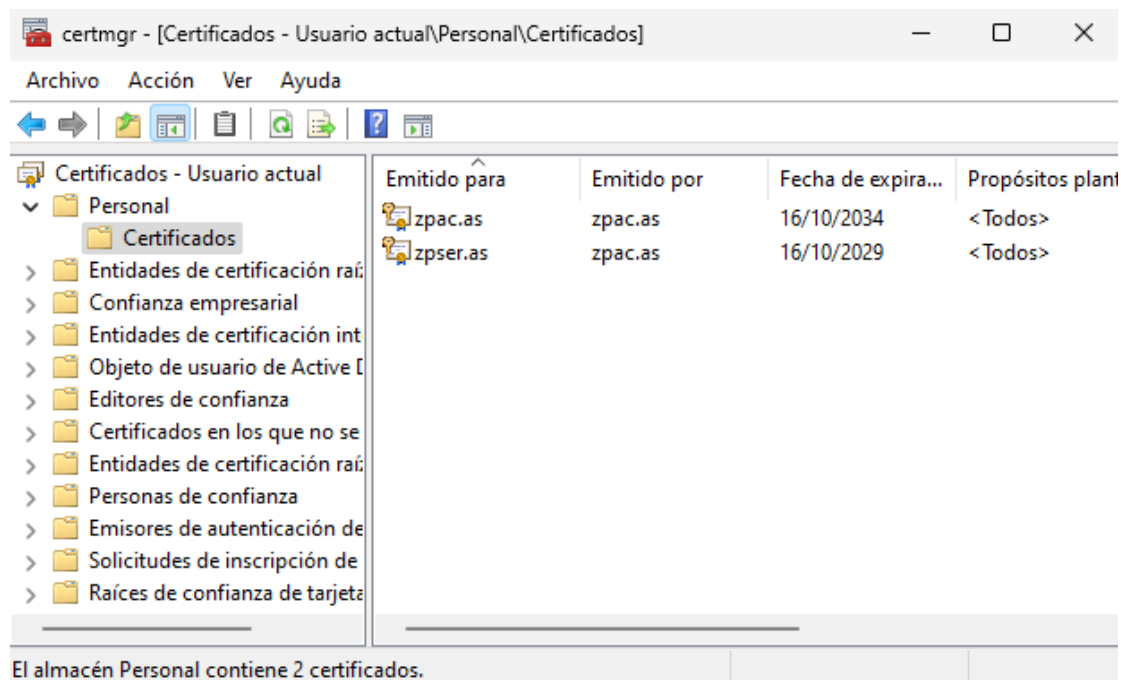
-DnsName: Especifica uno o más nombres DNS para colocar en la extensión "nombre alternativo del sujeto". Si no se especifica el parámetro -Subject, el primer nombre utilizado en el parámetro DnsName se asigna también como nombre el sujeto del certificado. Tras la ejecución del script, se puede comprobar con certmgr que la extensión "Nombre alternativo del titular" tiene los valores "Nombre DNS=zpser.as" "Nombre DNS=www.zpser.es" y "Nombre DNS=www.zpser.com".

-Signer: Especifica un objeto de tipo certificado y el cmdlet utiliza su clave privada asociada para firmar el nuevo certificado. El certificado indicado debe estar en el almacén de certificados personales y debe haber acceso de lectura a la clave privada del certificado.

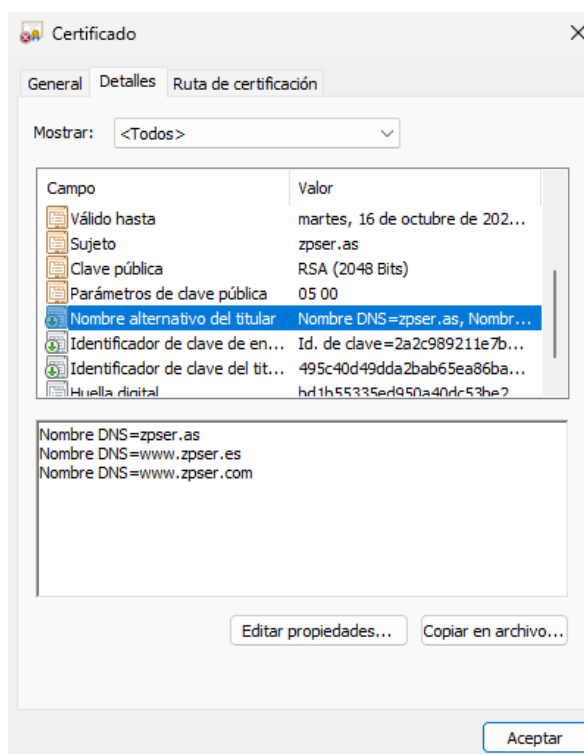
-TextExtensions: **NO SE HA UTILIZADO**

En el script anterior se podría haber utilizado oid=2.5.29.37 que representa "Enhanced Key Usage" y Cadena=1.3.6.1.5.5.7.3.1 que representa "Server Authentication". Esto permite restringir el uso del certificado al de autenticación de un servidor. Pero si nuestros programas no comprueban las extensiones no es útil incluirlas.

Utiliza la herramienta *certmgr.msc* para comprobar que el certificado emitido para *zpsr.as* está en el almacén de certificados "Personal", además del certificado de la autoridad certificadora que lo ha emitido.



Haz doble-clic en el certificado *zpsr.as* y en ventana Certificado selecciona la pestaña *Detalles*, y después selecciona el Campo "Nombre alternativo del titular". La imagen siguiente muestra el efecto del parámetro *-DnsName* al generar el certificado.



PARA GENERAR UN CERTIFICADO DE USUARIO

Repite los pasos realizados para generar un certificado de servidor. A continuación se muestran los parámetros utilizados con *New-SelfSignedCertificate* y el resultado de la ejecución.

```
New-SelfSignedCertificate -Type Custom `
-Subject "CN=zpusu.as" -DnsName "zpusu.as" `
-KeyAlgorithm RSA -KeyLength 2048 -KeySpec KeyExchange -KeyExportPolicy Exportable `
-KeyUsageProperty All -KeyUsage None `
-Provider "Microsoft Enhanced Cryptographic Provider v1.0" `
-NotBefore (Get-Date) `
-NotAfter (Get-Date).AddYears(5) `
-HashAlgorithm sha256 `
-Signer $cert `
-CertStoreLocation "Cert:\CurrentUser\My"
```

The screenshot shows the Windows PowerShell ISE interface. The command prompt displays the execution of the `New-SelfSignedCertificate` command. The output shows the certificate's thumbprint and subject information.

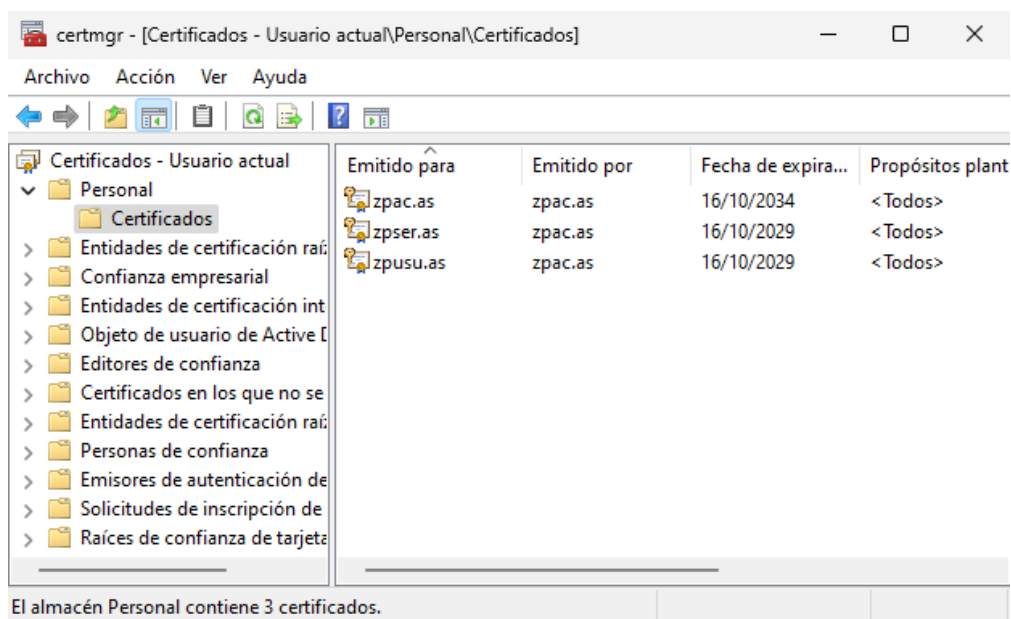
```
PS C:\Windows\system32> C:\Users\Admin\Desktop\PracticaB2.2\Certificados\CertificadoUsuario.ps1

PSParentPath: Microsoft.PowerShell.Security\Certificate::CurrentUser\My

Thumbprint                               Subject
-----
92BE6D6206D112F8E00559792989D5396D146704  CN=zpusu.as

PS C:\Windows\system32>
```

Con la herramienta *certmgr.msc* podrás comprobar que el certificado emitido para *zpusu.as* está en el almacén de certificados "Personal".



Si no se ve inicialmente, puedes pulsar el botón "Actualizar" (o pulsar F5) para que aparezca. Finalmente echa un vistazo a las propiedades de los certificados que has generado.

4. Exportación de Certificados

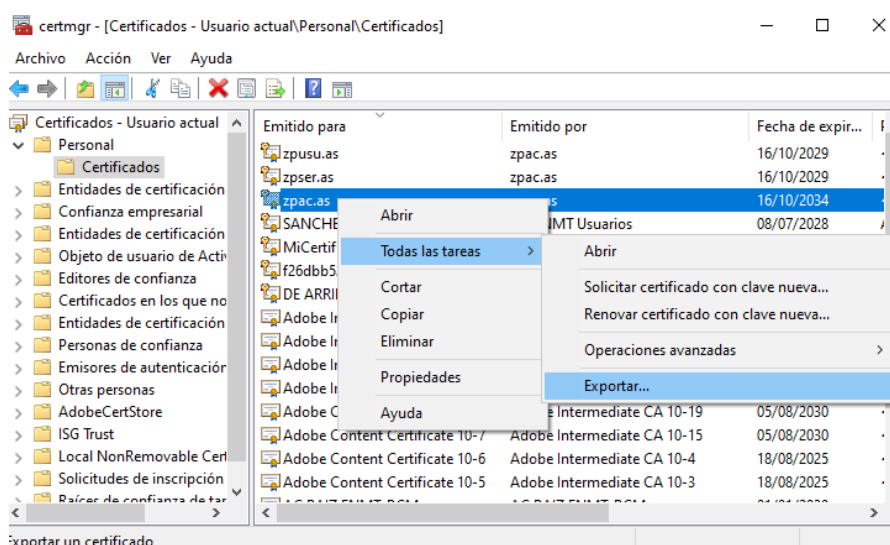
En la exportación de certificados hay que considerar dos posibilidades:

- 1.-Exportar solamente el certificado
- 2.-Exportar el certificado conjuntamente con su clave privada asociada.

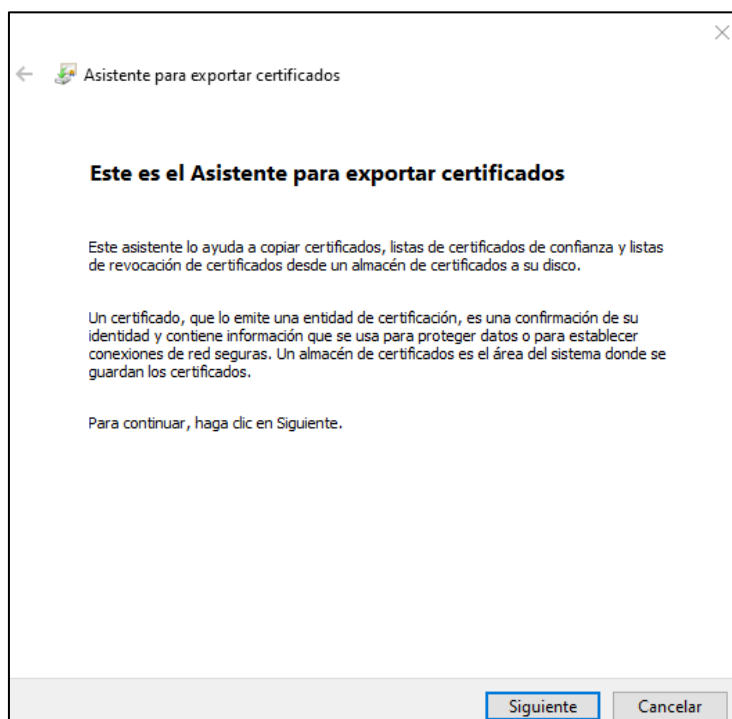
En esta práctica se considerarán las dos.

EXPORTACIÓN PARA LA AUTORIDAD CERTIFICADORA

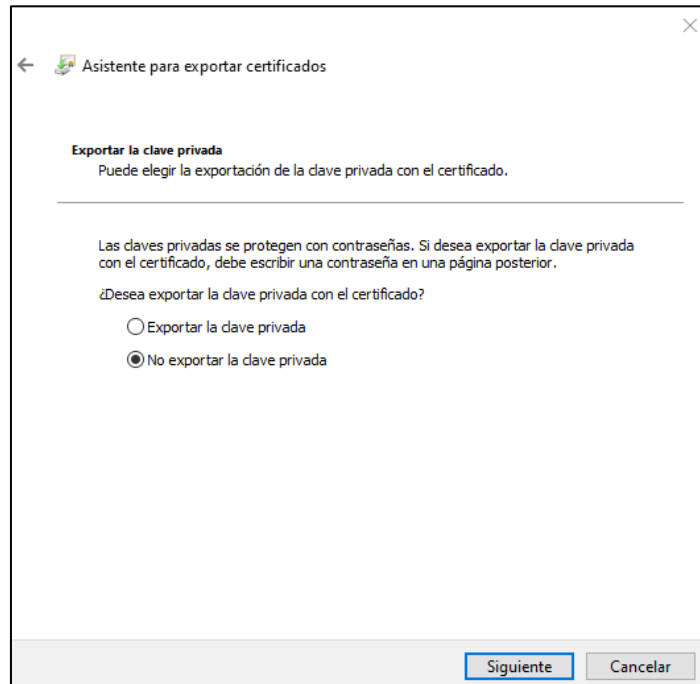
Utiliza la herramienta *certmgr.msc* para mostrar los certificados del almacén Personal y seleccionar el certificado de la autoridad certificadora *zpac.as*. Hacer clic-derecho para que se muestre la opción Exportar...



Entonces se abre al asistente para exportar certificados:

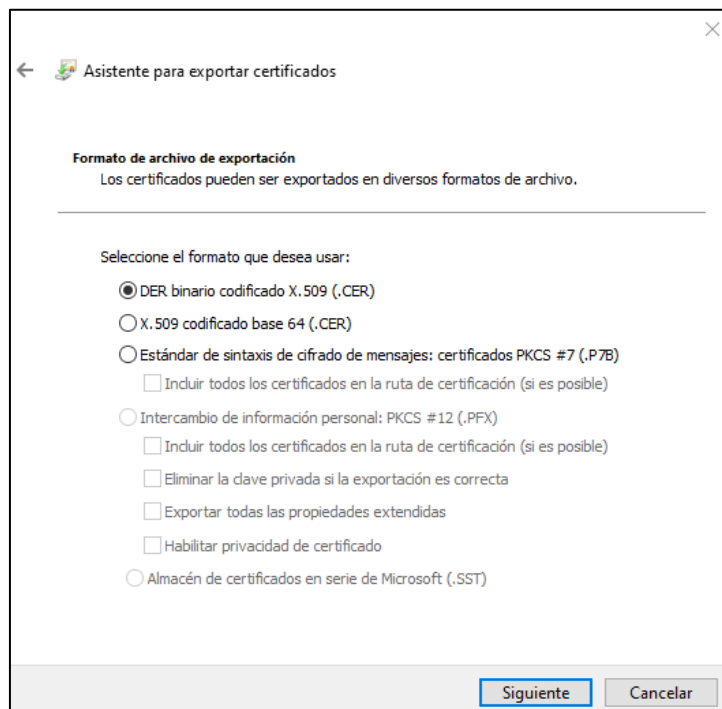


La primera opción a seleccionar es si desea exportar la clave privada con el certificado.



EXPORTAR CERTIFICADO SIN LA CLAVE PRIVADA

Selecciona NO exportar la clave privada. Entonces el asistente permite seleccionar 3 formatos para el certificado a exportar: DER, Base 64 y PKCS#7.



Utiliza el formato más común, que es el **formato DER**. Para dejar claro el formato en el que se ha exportado el certificado, se puede reflejar en el nombre del fichero. Por ejemplo: zpACas-DER.cer.

Ahora vuelve a exportar el certificado en **formato Base 64**. Por ejemplo usa el nombre de fichero zpACas-B64.cer para diferenciar esta exportación de la anterior. Comprueba que puedes abrir y ver este fichero con el bloc de notas, pero no puedes con el fichero exportado en formato DER.

Finalmente exporta el certificado en **formato PKCS#7**. Por ejemplo usa el nombre de fichero zpACas-PKCS7.p7b con la extensión estándar para este tipo de formato.

Comprueba que si haces doble clic sobre este fichero se abre automáticamente una nueva instancia de la aplicación *certmgr.msc* para visualizar el contenido del fichero. Pero si haces doble clic en los otros dos formatos se abre la ventana Certificado, que incluye un botón para la instalación del certificado en el almacén de certificados del sistema.

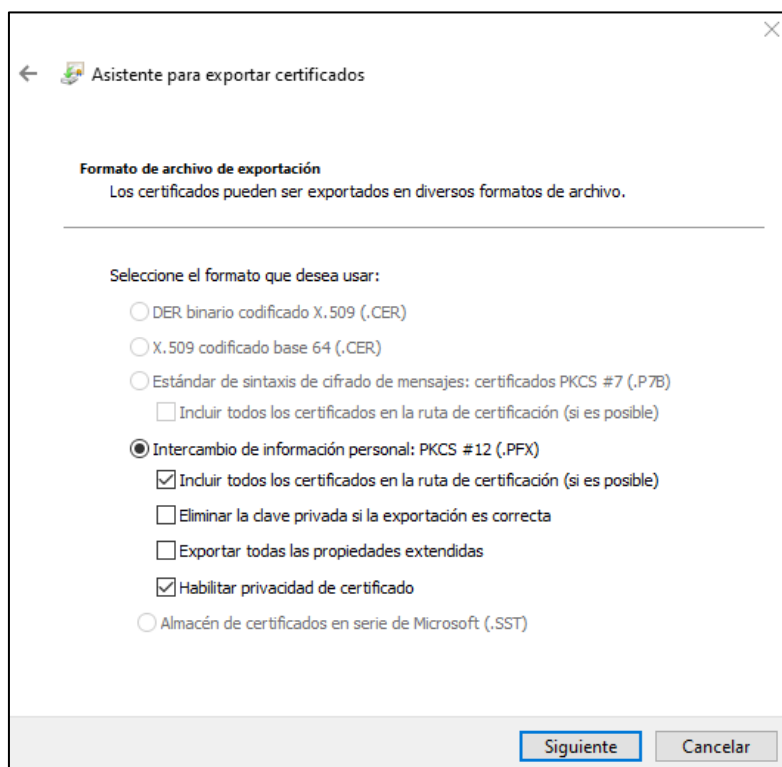
El formato PKCS#7 puede ser útil cuando hay que almacenar una cadena de certificados, pero en el caso de una autoridad certificadora, solo se almacena un fichero, por lo que no es útil.

Quédate con el certificado en formato DER que puedes renombrar por simplicidad a zpACas.cer.

EXPORTAR CERTIFICADO **CON** LA CLAVE PRIVADA

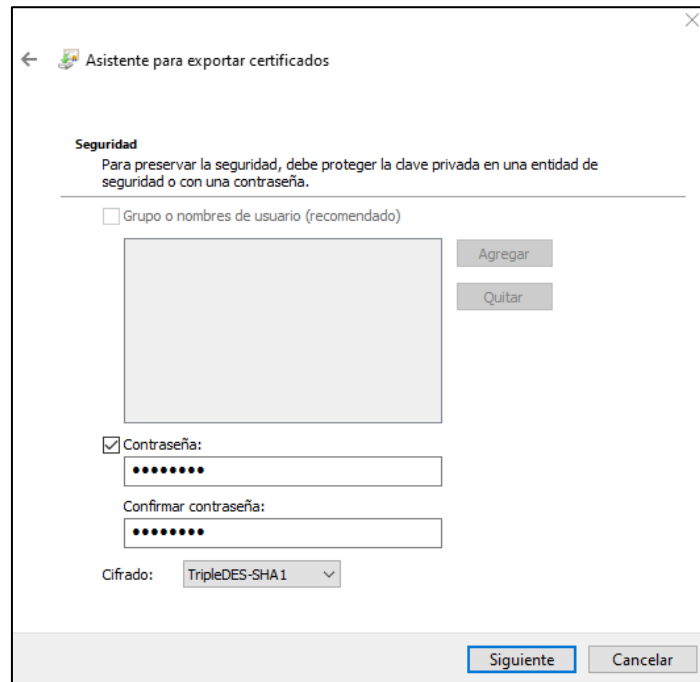
Una Autoridad Certificadora **solo** debe exportar la clave privada para disponer de un *backup*. El objetivo de esta exportación es obtener un fichero que contenga el certificado y su clave privada asociada. El formato utilizado es el PKCS#12.

Al seleccionar el formato PKCS#12, el asistente para exportar certificados permite varias opciones, y algunas de ellas ya están seleccionadas.



Se recomienda seleccionar **TODAS las opciones**, EXCEPTO Eliminar la clave privada si la exportación es correcta.

A continuación, para proteger la clave privada, el asistente va a cifrar el contenido del fichero con una clave simétrica que se derivará de una contraseña y realizar un resumen para poder comprobar posteriormente la integridad del fichero. El asistente muestra la ventana siguiente:



Como contraseña se recomienda proporcionar **conacpfx** (contraseña de la ac para el pfx).

Usando las contraseñas indicadas en el guion de esta práctica siempre existe la posibilidad de recordarlas consultando nuevamente el guion de la práctica.

Observar que para el cifrado se selecciona por defecto TripleDES-SHA1, pero se puede elegir también AES256-SHA256. Aunque la segunda opción sería la correcta, a veces da algún problema de compatibilidad. Por ello elige la primera.

Como nombre, para el fichero con ambas claves se recomienda usar zpACas.pfx.

EXPORTACIÓN PARA EL SERVIDOR

Realiza los mismos pasos que para la autoridad certificadora.

Exporta el certificado SIN clave privada solamente en formato DER binario. El fichero se puede denominar zpSERas.cer.

Exporta el certificado CON clave privada, usando como contraseña **conserpfx** (contraseña del servidor para el pfx). El fichero se puede denominar zpSERas.pfx.

EXPORTACIÓN PARA EL USUARIO

Realiza los mismos pasos que para la autoridad certificadora.

Exporta el certificado SIN clave privada solamente en formato DER binario. El fichero se puede denominar zpUSUas.cer.

Exporta el certificado CON clave privada, usando como contraseña **conusupfx** (contraseña del usuario para el pfx). El fichero se puede denominar zpUSUas.pfx.

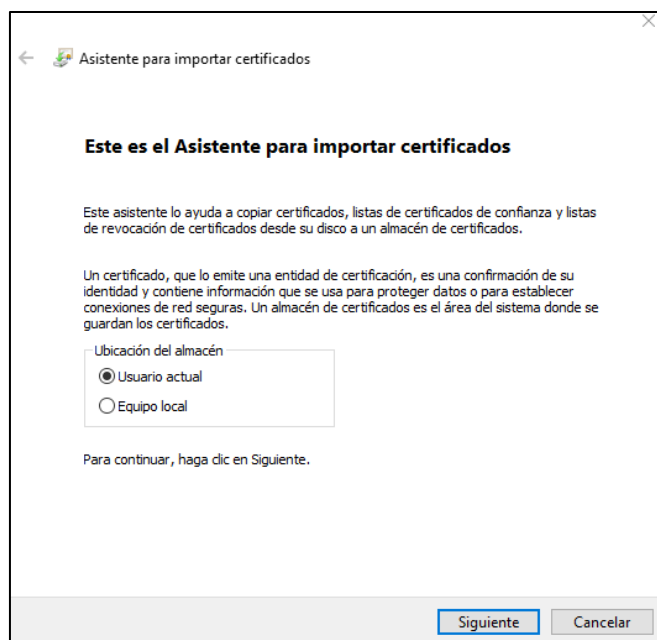
Verifica que todos los certificados se han creado de forma correcta y que ninguno ha provocado un error en la exportación.

5. Carga de los Certificados en el Almacén de Windows

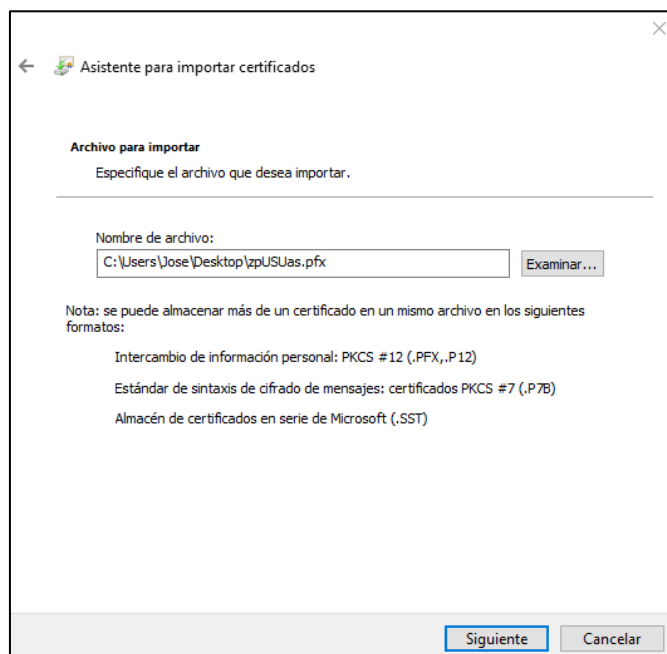
Se aconseja realizar la carga de certificados en la máquina física. Copia los ficheros zpACas.cer y zpUSUas.pfx de la máquina virtual a la máquina física, por ejemplo al directorio C:\TEMP\.

Para cargar los certificados en el almacén de certificados de Windows, en primer lugar hay que elegir si se desea cargar un certificado "clásico" (.cer) que contiene solamente la clave pública del sujeto o bien uno "completo" (.pfx) que contiene las claves pública y privada del sujeto.

Para cargar el certificado del usuario y su clave privada asociada, hacer doble clic con el ratón sobre el fichero zpUSUas.pfx, y aparece el asistente para la importación de certificados.



Selecciona *Usuario actual*. Un administrador de un computador también puede instalar el certificado como de *Equipo local*. El Asistente pide confirmación del fichero a importar.



Como el fichero zpUSUas.pfx contiene una clave privada protegida el asistente solicita la contraseña utilizada para protegerla.

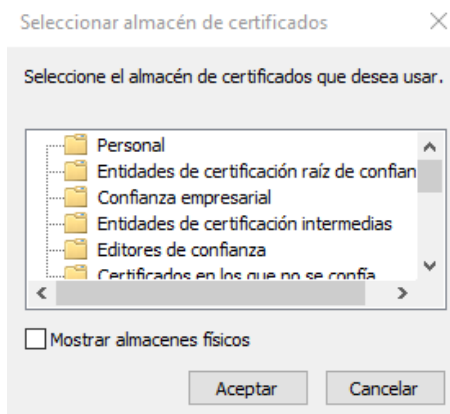
The screenshot shows the 'Asistente para importar certificados' window. The title bar includes a back arrow, a certificate icon, and the text 'Asistente para importar certificados'. The main heading is 'Protección de clave privada'. Below it, a message states: 'Para mantener la seguridad, la clave privada se protege con una contraseña.' A horizontal line separates this from the next instruction: 'Escriba la contraseña para la clave privada.' There is a text box labeled 'Contraseña:' containing ten dots. Below the text box is a checkbox labeled 'Mostrar contraseña'. A section titled 'Opciones de importación:' contains four checkboxes: 'Habilitar protección segura de clave privada...' (unchecked), 'Marcar esta clave como exportable...' (unchecked), 'Proteger la clave privada mediante security(Non-exportable) basada en virtualizado' (unchecked), and 'Incluir todas las propiedades extendidas.' (checked). At the bottom right are 'Siguiente' and 'Cancelar' buttons.

Si se han seguido las indicaciones de la práctica la contraseña será **conusupfx**. No habilites la protección segura de clave privada, marca la clave privada como exportable e incluye las propiedades extendidas del certificado.

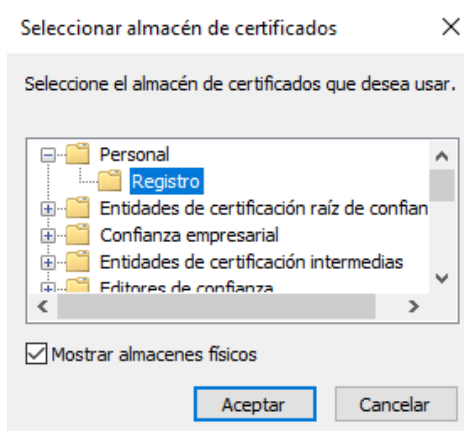
Ahora hay que elegir el almacén de certificados en el que se desea realizar la importación. Se puede permitir que el asistente seleccione automáticamente el almacén o bien elegirlo. Utilizar esta segunda opción como muestra la pantalla siguiente:

The screenshot shows the 'Asistente para importar certificados' window at the 'Almacén de certificados' step. The title bar is the same as the previous window. The heading is 'Almacén de certificados'. Below it, a message states: 'Los almacenes de certificados son las áreas del sistema donde se guardan los certificados.' A horizontal line separates this from the next instruction: 'Windows puede seleccionar automáticamente un almacén de certificados; también se puede especificar una ubicación para el certificado.' There are two radio button options: 'Seleccionar automáticamente el almacén de certificados según el tipo de certificado' (unselected) and 'Colocar todos los certificados en el siguiente almacén' (selected). Below the selected option is a text box labeled 'Almacén de certificados:' and an 'Examinar...' button. At the bottom right are 'Siguiente' and 'Cancelar' buttons.

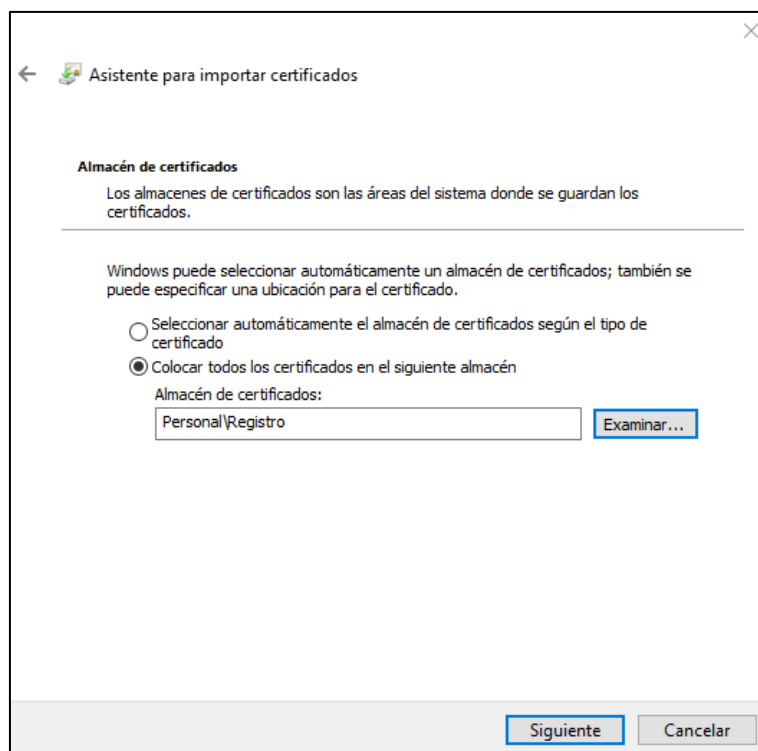
Al pulsar el botón *Examinar...* aparece la ventana "Seleccionar almacén de certificados".



Selecciona la opción *Mostrar los almacenes físicos*, despliega los almacenes físicos del almacén Personal, y selecciona el único almacén físico disponible, tal como se muestra en la figura siguiente:

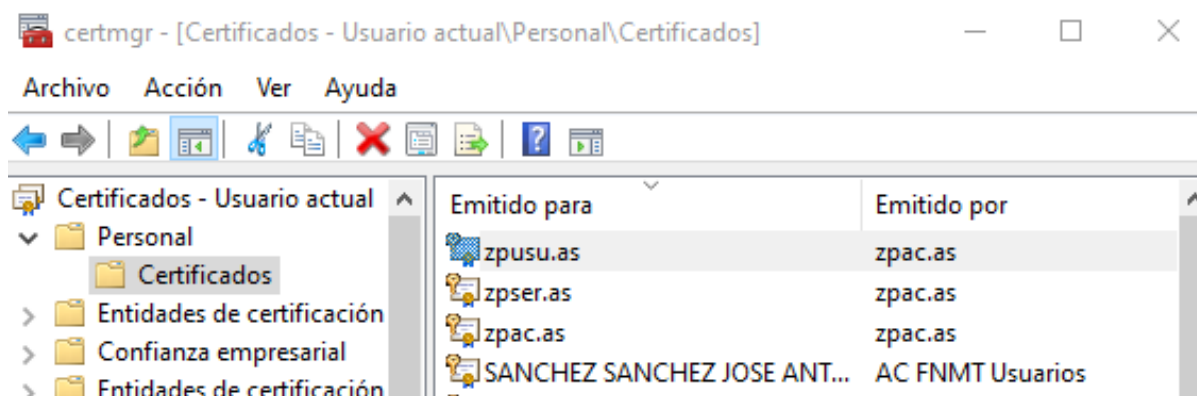


Al pulsar el botón *Aceptar*, el asistente de importación muestra la siguiente información:



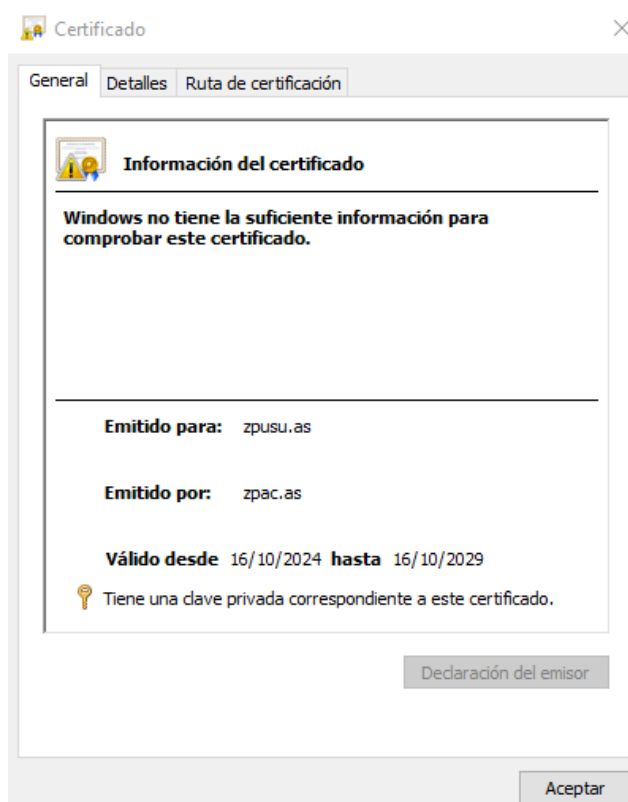
Pulsar el botón *Siguiente* y *Finalizar* el proceso de importación.

Comprobar con la herramienta *certmgr* que el certificado ha sido importado con éxito.



Observa que el proceso de carga también ha cargado el certificado de la autoridad certificadora de *zpusu.as* en el almacén de certificados personales. Esto no es correcto. Elimina el certificado *zpac.as*.

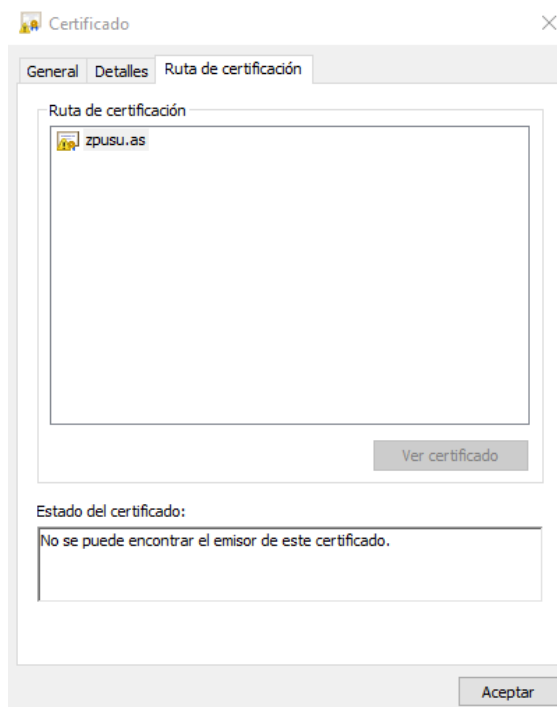
Para ver las propiedades del certificado haz doble clic sobre el certificado y aparece la ventana siguiente, que muestra la pestaña "General" con información del certificado.



Observa el mensaje que aparece en la parte inferior del cuadro de información. Se informa que hay una clave privada correspondiente al certificado.

Observa también el mensaje que hay en la parte superior del cuadro informativo, que indica que Windows no tiene la suficiente información para comprobar este certificado (al haber eliminado *zpAC.as*). El problema está relacionado con la cadena de certificados necesaria para validar el certificado de *zpusu.as*. Lo analizamos a continuación.

Comprobar la ruta de certificación y el estado de este certificado, seleccionando la pestaña "Ruta de certificación":



Observar que no hay una ruta disponible, y en relación al estado, el gestor de certificados no puede encontrar al emisor del certificado.

Cargar el certificado de la autoridad certificadora zpACas.cer, emisora del certificado de zpusu.as, en el almacén denominado "*Entidades de certificación raíz de confianza*". Para ello hacer doble clic sobre el fichero zpACas.cer y se abre la ventana siguiente:

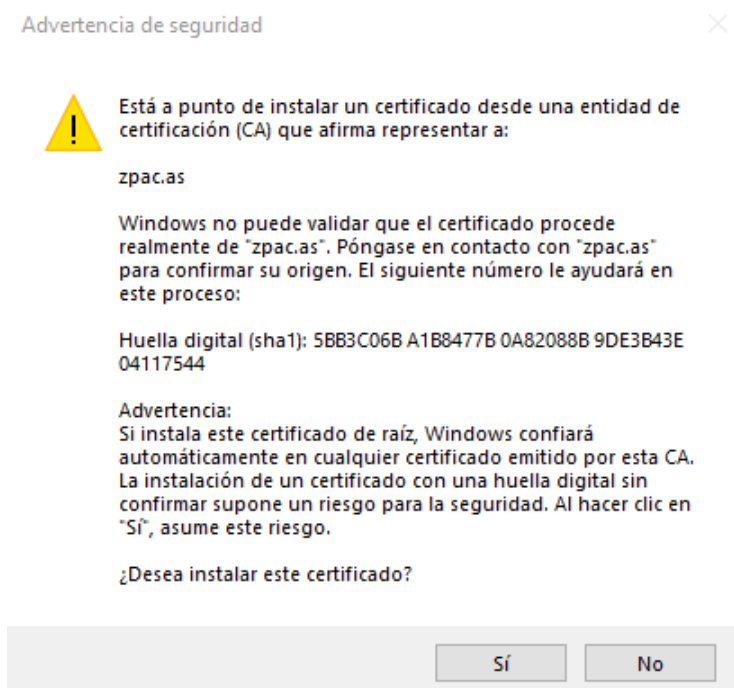


Observar que en la parte inferior del cuadro informativo no se indica que hay una clave privada asociada al certificado, lo cual es correcto, pues estamos usando un fichero .cer.

Pulsar el botón *Instalar certificado...*

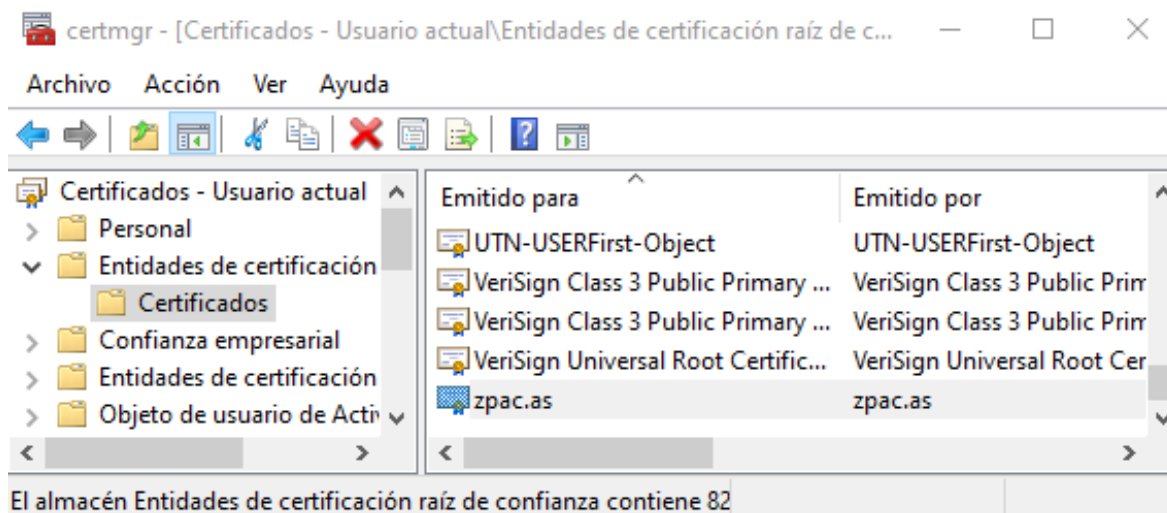
En el Asistente para importar certificados, seleccionar *Usuario actual* y el almacén “*Entidades de certificación raíz de confianza*”. No elegir el almacén físico, dejando que elija el asistente.

El asistente muestra la ventana siguiente:



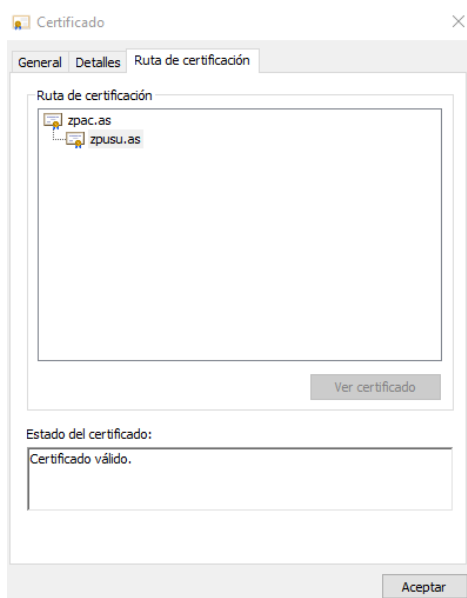
Al aceptar la instalación de un nuevo certificado raíz, creamos un nuevo anclaje de confianza y nuestro computador confiará en todos los certificados emitidos por la autoridad certificadora zpac.as.

Comprueba con *certmgr* que se ha importado correctamente y que aparece al final de todos los certificados (para eso lo llamamos zp..., z para que aparezca al final y sea fácilmente localizable y porque se generó con PowerShell). Puede que tengas que pulsar el botón actualizar (flecha giratoria a la derecha o tecla F5) para que aparezca el nuevo certificado instalado.



No es sencillo ver donde almacena Windows estos certificados, pues no está oficialmente documentado. Se supone que residen en algún directorio del sistema de archivos del SO.

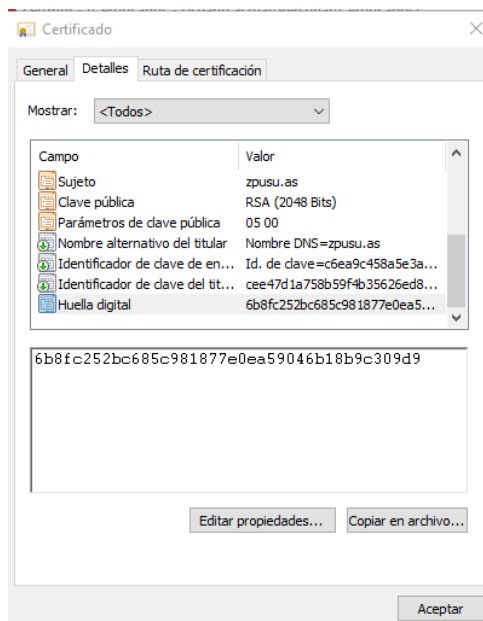
Ahora, en la herramienta *certmgr* abre la carpeta de certificados personales, pulsa en zpusu.as y selecciona la pestaña "Ruta de certificación". Aparece la siguiente ventana.



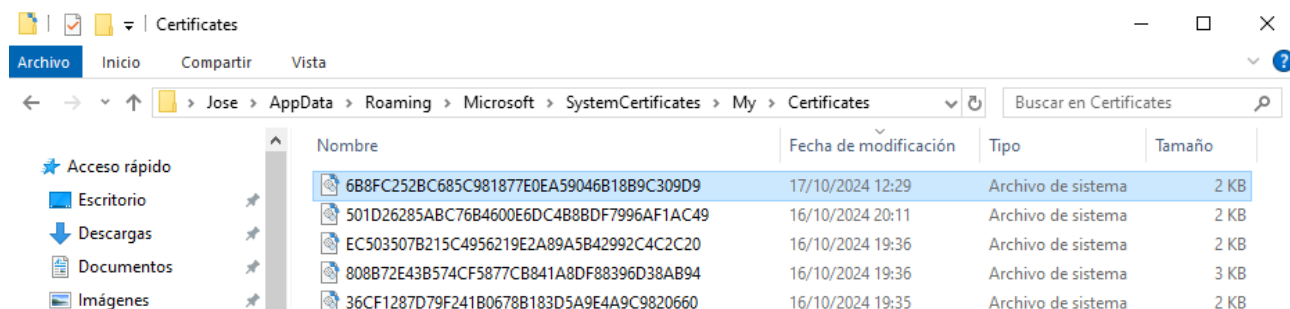
Comprueba como ahora el certificado tiene una ruta de certificación definida y el sistema considera que el certificado es válido.

ALMACENAMIENTO DE LOS CERTIFICADOS EN EL SISTEMA OPERATIVO

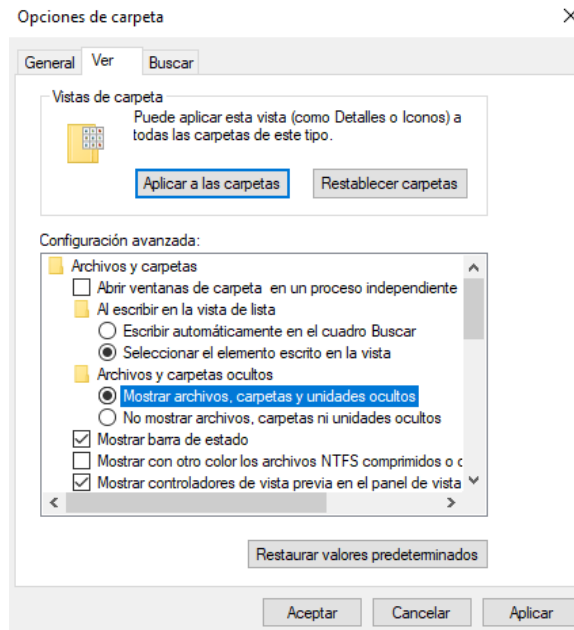
Con la herramienta *certmgr* muestra el campo del certificado de *zpusu.as* denominado Huella digital, tal como se muestra en la figura siguiente:



Comprobar que el certificado ha sido almacenado en el directorio y fichero que se pueden ver en la ventana siguiente (adaptar la ruta al computador utilizado), como norma general será en el directorio: *C:\Users\Usuario\AppData\Roaming\Microsoft\SystemCertificates\My\Certificates*.

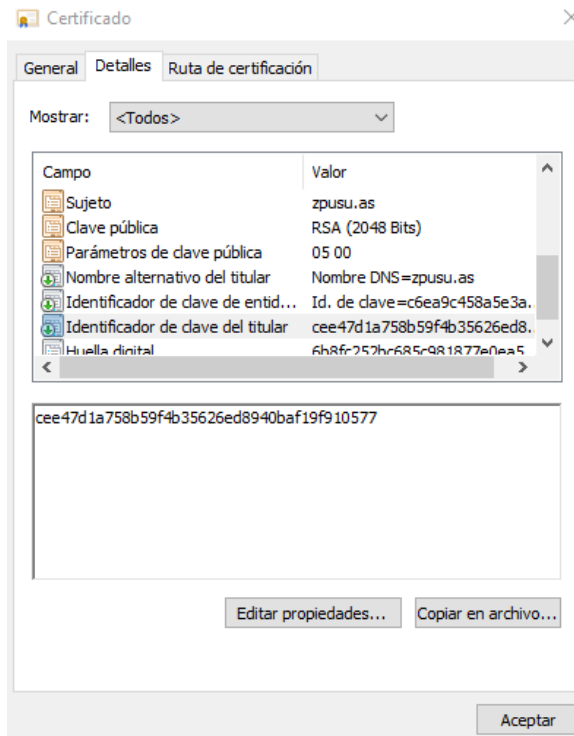


Para llegar a ese directorio debes permitir que el explorador de Windows muestre los archivos, carpetas y unidades ocultos (aunque si la ruta es exacta lo abrirá). Para ello, en una ventana del explorador de archivos, activa la vista de ficheros y carpetas ocultos desde las opciones de carpeta.

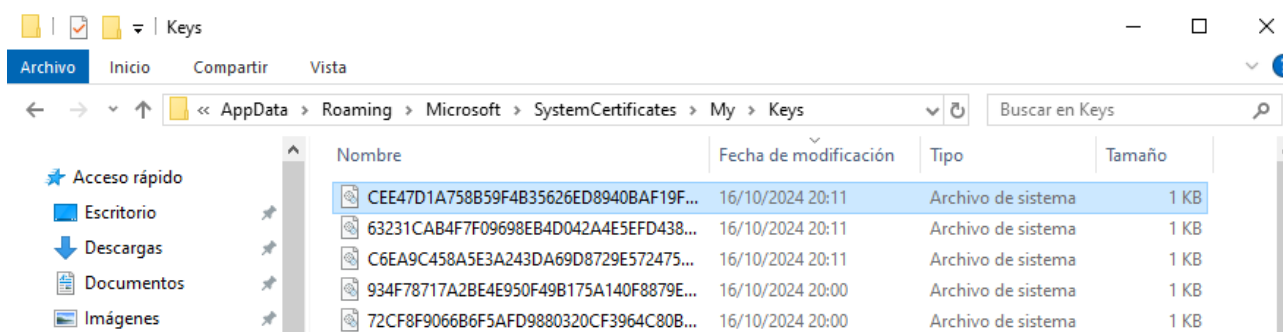


Comprueba que la huella digital del certificado coincide con el nombre del fichero el que se almacena el certificado.

Con la herramienta *certmgr* muestra el campo del certificado de *zpusu.as* denominado *Identificador de clave del titular*, tal como se muestra en la figura siguiente:



Comprueba que se ha creado una clave privada en el directorio predeterminado para contener las claves del usuario justo al mismo tiempo, y que el Identificador de clave del titular coincide con el nombre del fichero en el que se almacena la clave, tal como se muestra en la ventana siguiente, cuyo directorio es: *C:\Users\Usuario\AppData\Roaming\Microsoft\SystemCertificates\My\Keys*



No hay documentación sobre los mecanismos que usa el SO para almacenar las claves. No obstante, observar la coincidencia de fechas, horas y minutos en la creación de los ficheros con la de importación del certificado.

En la ventana de la herramienta *certmgr* eliminar el certificado. Comprobar que también desaparece el fichero correspondiente del directorio en el que se almacenan los certificados. **Pero la clave privada RSA asociada al certificado no se elimina automáticamente de los directorios correspondientes.** Si no deseamos retener las claves privadas en el sistema hay que borrar sus ficheros manualmente.

Generalmente, el usuario debe despreocuparse del almacenamiento de las claves privadas asociadas a los certificados, permitiendo que el sistema operativo gestione su almacenamiento.

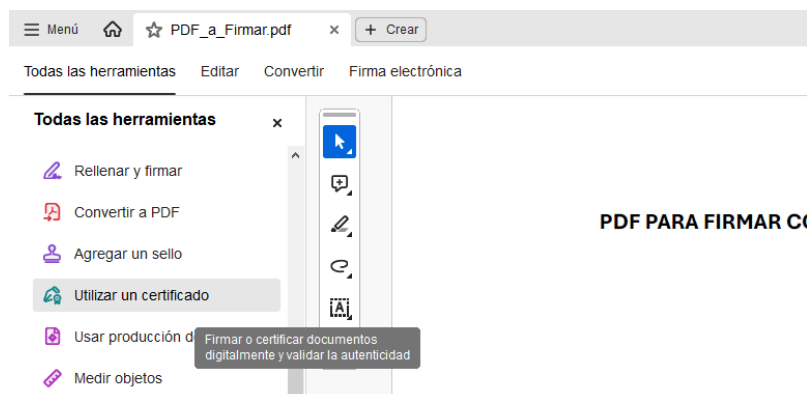
Pero entonces, la seguridad de las claves privadas de cada usuario, depende de la seguridad del sistema de ficheros y de la contraseña del usuario para el acceso al sistema operativo.

6. Firma de PDF con Certificado Personal

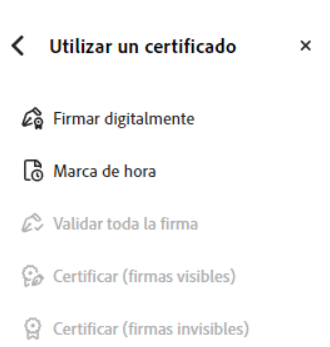
En esta parte haremos uso del certificado digital expedido por la FNMT que hemos solicitado previamente y que tenemos incorporado en nuestro almacén de certificados de usuario.

Esta parte de la práctica, se presupone que se puede realizar sobre la máquina anfitriona, de todas formas, podemos realizarla también sobre la máquina virtual. Si esta última y no lo tienes instalado, instala *Adobe Acrobat Reader* en el equipo. Para ello, entra en: <https://get.adobe.com/es/reader/> y procede a realizar la descarga e instalación del software que utilizaremos para realizar la firma con el certificado personal.

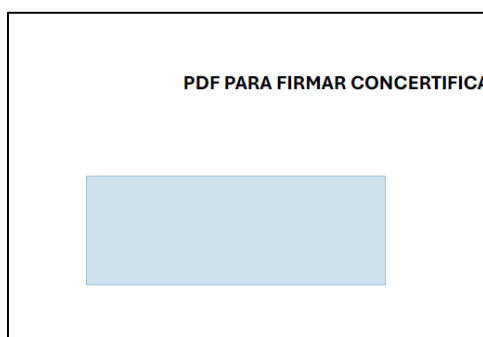
En cualquier caso, para firmar el PDF deberás abrir el pdf en el lector *Adobe Acrobat Reader* y acceder a las herramientas que nos ofrece. Dentro de ellas, seleccionaremos la opción “Utilizar un certificado”:



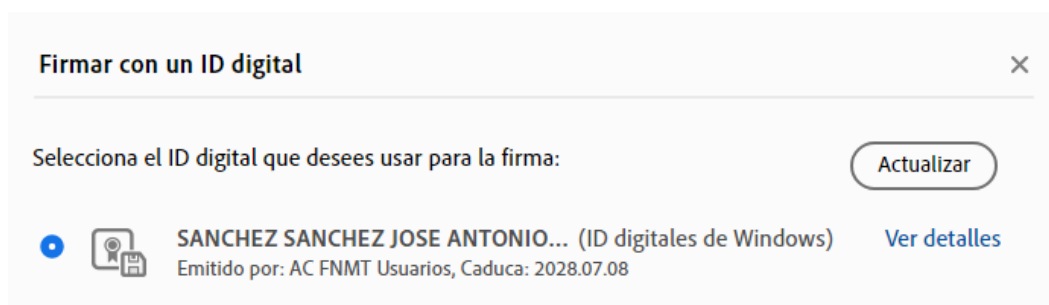
Una vez que pulsemos sobre esa opción, veremos que se abre un submenú que nos ofrece la posibilidad de firmar digitalmente.



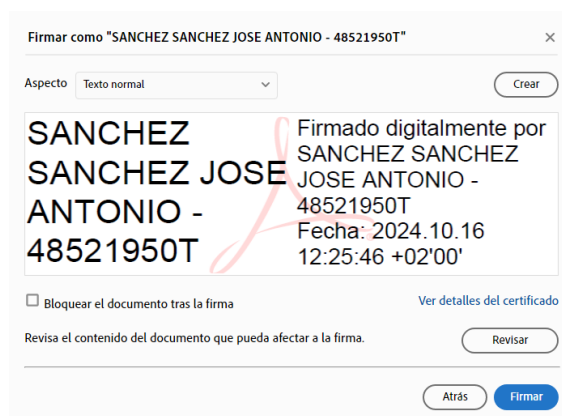
Selecciona esta opción para proceder con la firma digital. Para poder realizarla y una vez que pulsemos sobre “Firmar Digitalmente”, vemos que el cursor cambia en el área del documento (se establece una cruz). Esta cruz indicará la posición sobre la que realizar la firma. Pincha y arrastra el cursor en el área en la que quieras firmar.



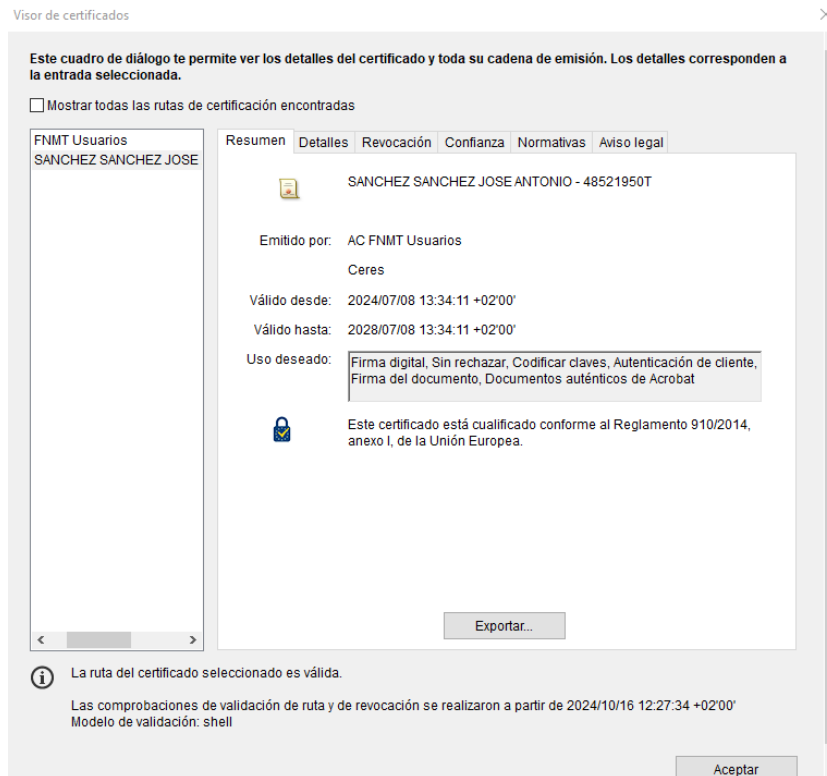
Automáticamente, nos aparecerán los certificados disponibles para proceder con la firma. En mi caso, se muestra el emitido por la FNMT que tengo ya importado en mi almacén de certificados.



Selecciona este y pulsa en el botón *Continuar*. Aparecerá ahora la pantalla de la firma en la que podemos configurarla.



En esta pantalla vamos a verificar los detalles del certificado antes de proceder a la firma. Pincha sobre “Ver detalles del certificado”.

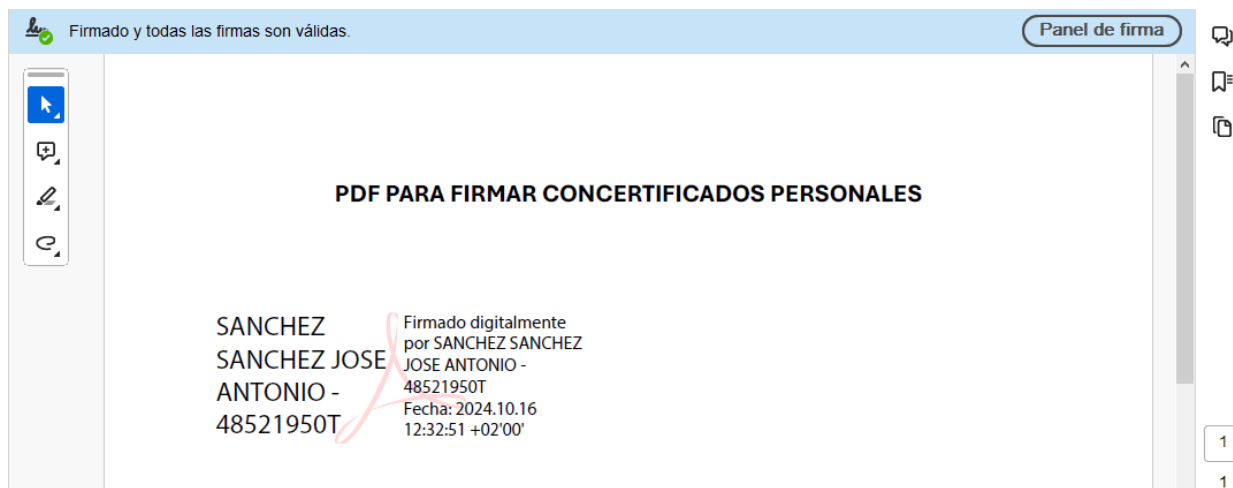


Como puedes ver en el panel izquierdo, puedes obtener los detalles del certificado, tanto del usuario como de la entidad emisora del mismo (FNMT). Verifica las opciones que nos ofrecen las diferentes pestañas (*Detalles*, *Revocación*, *Confianza*, *Normativas* y *Aviso Legal*). A continuación, pincha sobre los detalles de la entidad emisora del certificado “*FNMT Usuarios*”. Realiza de nuevo la misma revisión de las pestañas disponibles donde se puede obtener información relativa al certificado.

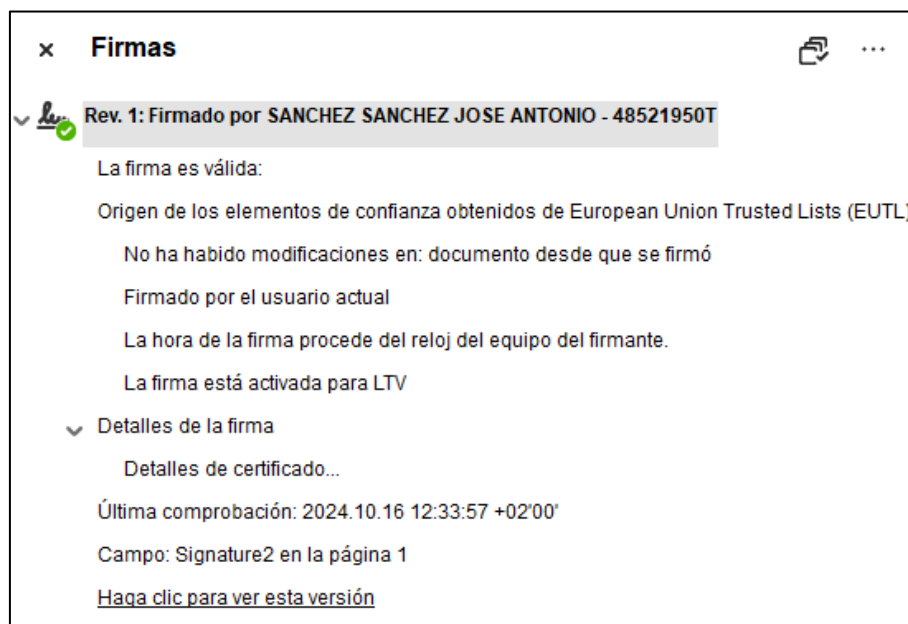
Desde esta misma pantalla, podríamos proceder a exportar el certificado, pero no lo haremos por el momento.

Una vez que cierres la pestaña y vuelvas a las propiedades de la firma, verifica que la casilla “*Bloquear el documento tras la firma*” no está marcado ya que, esto haría que no pudiéramos modificar (ni firmar por terceros) el documento.

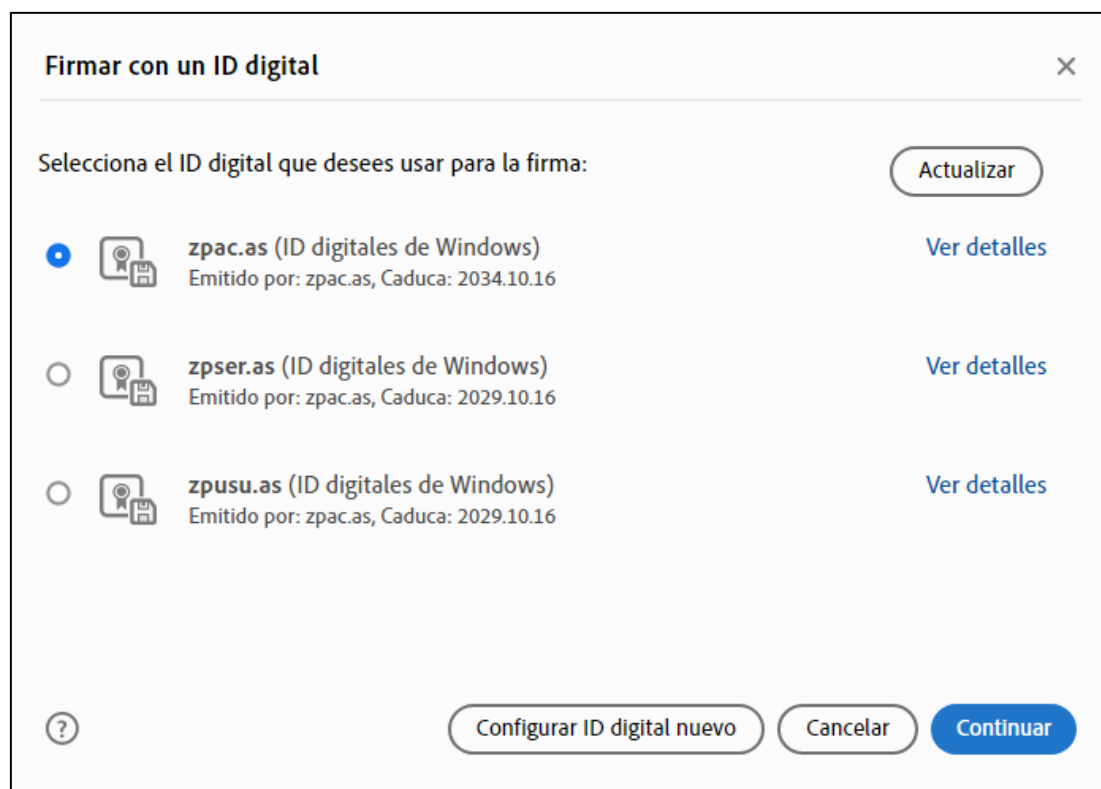
Pincha en “*Firmar*” para proceder con la firma del PDF. Nos pedirá que almacenemos el fichero, procede a guardarlo de nuevo (puedes variar el nombre) para que la firma se realice.



Como podemos ver, en la imagen superior, el documento ha sido firmado y nos aparece un mensaje superior en el que nos dice que está firmado y que las firmas son válidas. Para verificar esto accede al *Panel de Firma* que tienes en la parte superior derecha. En este punto podrás ver la información de la firma, así como revisar de nuevo los certificados empleados.



A continuación, se pide que se realice una nueva firma (en el mismo documento) haciendo uso de alguno de los certificados creados en anteriores puntos de esta práctica:



Una vez que ya conocemos la firma en PDF haciendo uso de un certificado personal, **Configura un nuevo ID digital** (puedes verlo en la figura superior). Selecciona para ello “*Crea una ID Digital Nueva*” y guarda el certificado en el almacén de certificados de Windows. Establece los valores que puedes observar a continuación para el nuevo ID digital.

Crear un ID digital firmado automáticamente

Introduce la información de identidad que se usará para crear el ID digital firmado automáticamente.

Los ID digitales firmados automáticamente por individuos no ofrecen la seguridad de que la información de identidad sea válida. Por este motivo, es posible que no se acepten en algunos casos.

Nombre

Unidad organizativa

Nombre de la organización

Dirección de correo electrónico

País o región

Algoritmo clave

Uso de ID digital

[?](#)

[Atrás](#) [Guardar](#)

Verifica que el certificado se ha creado correctamente y que puedes firmar con él. Observa también las propiedades del mismo ("Ver detalles") para ver quien lo emite, hasta cuándo es válido, etc. Comprueba todas las pestañas que se ofrece en los detalles al igual que hiciste con anteriores certificados.

Firmar con un ID digital

Selecciona el ID digital que desees usar para la firma:

[Actualizar](#)

MiCertificado (ID digitales de Windows)
Emitido por: MiCertificado, Caduca: 2029.10.16

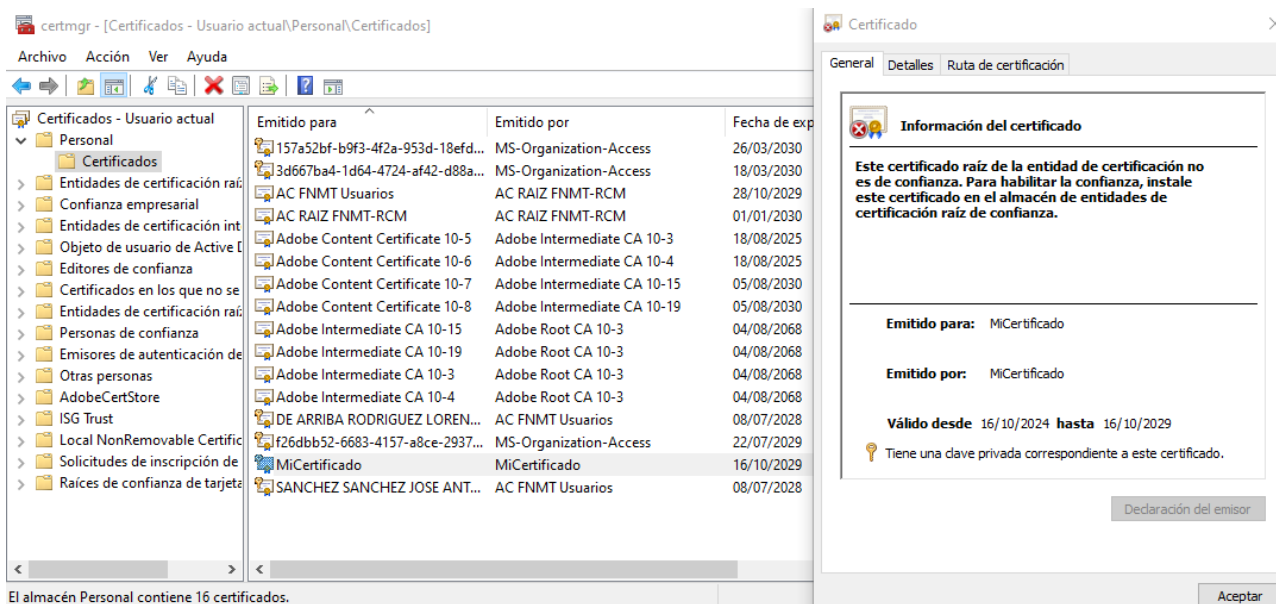
[Ver detalles](#)

Antes de firmar, cambia el aspecto de la firma para que se muestre como la imagen inferior:

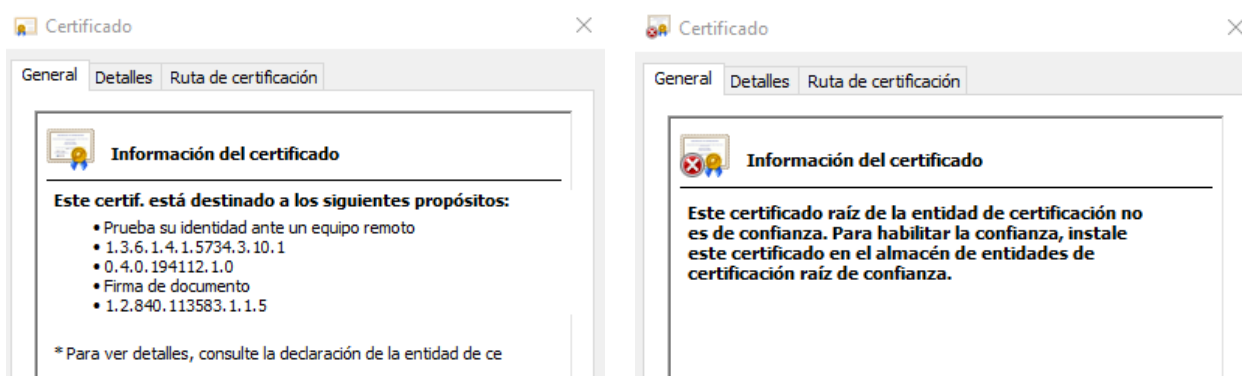


A continuación, **bloquea el documento** antes de la firma y procede a firmarlo con el certificado creado anteriormente. Verifica en este momento el panel de firma para verificar si las firmas realizadas sobre el documento son válidas.

Por último, **localiza el nuevo certificado** creado en la **herramienta de gestión de certificados** de usuario que hemos visto con anterioridad y comprueba sus **propiedades**.



Como puedes observar en las propiedades, la información del certificado nos expone el problema del uso de este tipo de certificados, ya que no sería de confianza (inicialmente) aunque podríamos instalarlo para que lo fuera. Comprueba que tu certificado de la FNMT si es de confianza y las acciones para las que se usa, comparándolo con este. Ten en cuenta que este tipo de certificados no serán validos ante una entidad certificadora real, por lo que, no sirven para firmar documentos de forma oficial.



7. Anexo A: Creación de Certificados con Makecert (Opcional)

Si finalizas la práctica y lo consideras necesario, puedes realizar este anexo, aunque nunca será evaluable ni exigible al alumnado, ya que es un apartado **opcional**.

El programa makecert permite crear certificados y claves privadas para una autoridad certificadora y con ellos crear certificados para servidores y clientes, y en general para usuarios.

Microsoft, aunque mantiene funcional la herramienta makecert (que está en desuso desde Julio de 2024), recomienda que se use el cmdlet `New-SelfSignedCertificate` del entorno PowerShell. En este anexo se explica la creación de certificados con makecert como una alternativa a `New-SelfSignedCertificate`.

La ayuda sobre el programa makecert.exe está integrada en la ayuda de Visual Studio. En la pestaña de índice o en la de contenido buscar Makecert.exe.

La ayuda en Internet sobre makecert.exe está disponible en:

<https://docs.microsoft.com/es-es/windows/win32/seccrypto/makecert>

El propio programa makecert proporciona ayuda así:

makecert -? Muestra la ayuda de las opciones básicas

makecert -! Muestra la ayuda de las opciones extendidas

El programa makecert puede crear un certificado directamente en uno de los almacenes lógicos del sistema operativo (físicamente residen en directorios del sistema operativo) y/o crearlo en un fichero. En esta práctica se recomienda crear los certificados exclusivamente en ficheros. Posteriormente se importa el certificado en un almacén lógico desde el fichero.

El programa makecert necesita claves para generar los certificados, que pueden estar en el almacén de claves del sistema operativo o en ficheros. Si no hay claves disponibles, makecert puede crear automáticamente las claves y guardarlas en el almacén de claves del SO y/o en ficheros. En esta práctica se recomienda permitir la creación automática de claves y guardarlas exclusivamente en ficheros.

Hay que crear tres certificados:

- 1) Un certificado auto-firmado que correspondería al certificado raíz de una Autoridad Certificadora.
- 2) Un certificado de servidor que debe ser firmado usando la clave privada asociada al certificado de la Autoridad Certificadora.
- 3) Un certificado de cliente que debe ser firmado usando la clave privada asociada al certificado de la Autoridad Certificadora.

NOTA: Los prefijos de los nombres de las entidades y de los ficheros, como zmAC.as, tienen esta racionalidad:

z → Para que los certificados aparezcan al final de los almacenes y sean fácilmente localizables

m → Para indicar que se han generado con makecert, o p para indicar generados con powershell

PARA CREAR UN CERTIFICADO RAIZ:

Se recomienda crear la orden en un archivo zmACas.bat que permita corregir cómodamente y documentar las opciones a utilizar que son, como mínimo, las siguientes:

- Nombre del sujeto del certificado; usar -n "CN=zmAC.as"
- Creación de un certificado auto-firmado; usar -r
- Permitir que la clave privada generada sea exportable; usar -pe
- Tipo del certificado; usar -cy authority
- Número de serie del certificado; usar -# 1
- Longitud de la clave del sujeto; usar -len 2048
- Algoritmo de resumen para firmar el certificado; usar -a sha256
- Nombre del fichero que contendrá la clave privada generada; usar -sv "zmACas.pvk"
- Nombre del fichero que contendrá el certificado; usar zmACas.cer

Para identificar inequívocamente los certificados en un futuro uso cruzado entre alumnos, es mejor utilizar como nombre del sujeto zmACnombrealumno, por ejemplo zmACalicia o zmACbenito. Esto también se puede aplicar a los nombres de los ficheros.

Al ejecutar makecert se pide una contraseña para proteger la clave privada de la autoridad certificadora que se crea automáticamente y que se debe almacenar en un fichero. Usar **conac**. A continuación, makecert pide esta contraseña para generar el fichero con la clave privada.

Hacer doble clic sobre el fichero zmACas.cer para ver la información general y los detalles del certificado, así como la ruta de certificación (un solo elemento).

Observar que en el directorio aparece el fichero zmACas.pvk que contiene la clave privada de la autoridad certificadora.

PARA CREAR UN CERTIFICADO DE SERVIDOR:

Se recomienda crear la orden en un archivo zmSERas.bat que permita corregir cómodamente y documentar las opciones a utilizar que son, como mínimo, las siguientes:

- Nombre del sujeto del certificado; usar -n "CN=zmSER.as" (o mejor zmSERnombrealumno)
- Permitir que la clave privada generada sea exportable; usar -pe
- Tipo del certificado; usar -cy end
- Nombre del fichero que contiene el certificado del emisor; usar -ic "zmACas.cer"
- Nombre del fichero que contiene la clave privada del emisor; usar -iv "zmACas.pvk"
- El tipo del clave del sujeto; usar -sky Exchange
- Longitud de la clave del sujeto; usar -len 2048
- Algoritmo de resumen para firmar el certificado; usar -a sha256
- Nombre del fichero que contendrá la clave privada generada; usar -sv "zmSERas.pvk"
- Nombre del fichero que contendrá el certificado; usar zmSERas.cer

Al ejecutar makecert se pide una contraseña para proteger la clave privada del servidor que debe crear automáticamente y que se debe almacenar en un fichero. Usar por ejemplo **conser**. A continuación makecert pide esta contraseña para generar el fichero con la clave privada.

Finalmente, makecert pide la contraseña de la clave privada del emisor del certificado (la autoridad certificadora) para firmar el certificado. Esta contraseña es **conac**.

Hacer doble clic sobre el fichero zmSERas.cer para ver la información general y los detalles del certificado, así como la ruta de certificación (dos elementos).

PARA CREAR UN CERTIFICADO DE CLIENTE:

Se recomienda crear la orden en un archivo zmCLlas.bat que permita corregir cómodamente y documentar las opciones a utilizar que son, como mínimo, las siguientes:

- Nombre del sujeto del certificado; usar -n "CN=zmCLI.as" (o mejor zmCLInombrealumno)
- Permitir que la clave privada generada sea exportable; usar -pe
- Tipo del certificado; usar -cy end
- Nombre del fichero que contiene el certificado del emisor; usar -ic "zmACas.cer"
- Nombre del fichero que contiene la clave privada del emisor; usar -iv "zmACas.pvk"
- El tipo del clave del sujeto; usar -sky Exchange
- Longitud de la clave del sujeto; usar -len 2048
- Algoritmo de resumen para firmar el certificado; usar -a sha256
- Nombre del fichero que contendrá la clave privada generada; usar -sv "zmCLlas.pvk"
- Nombre del fichero que contendrá el certificado; usar zmCLlas.cer

Al ejecutar makecert se pide una contraseña para proteger la clave privada del cliente que debe crear automáticamente y que se debe almacenar en un fichero. Usar por ejemplo **concli**. Posteriormente makecert pide esta contraseña para generar el fichero con la clave privada.

Finalmente, makecert pide la contraseña de la clave privada del emisor del certificado (la autoridad certificadora) para firmar el certificado. Esta contraseña es **conac**.

Hacer doble clic sobre el fichero zmCLlas.cer para ver la información general y los detalles del certificado, así como la ruta de certificación (dos elementos).

CONVERSION DE LOS CERTIFICADOS:

Los certificados anteriores (.cer) NO incluyen la clave privada del sujeto para el que se ha emitido el certificado. Pero en muchas ocasiones es necesario que el sujeto disponga de su pareja de claves (pública y privada) conjuntamente.

Para ello se puede usar el programa pvk2pfx.exe que añade la clave privada (.pvk) al certificado (.cer) generando un nuevo certificado del tipo PFX (.pfx) que implementa el estándar PKCS#12 (*Personal Information Exchange Syntax Standard*). **Observar que denominar como certificado a un fichero PFX que incluye una clave privada es inadecuado, aunque habitual.**

Como nombres de estos nuevos certificados a crear se sugiere utilizar: zmCLlas.pfx y zmSERas.pfx. NO generar un .pfx para la Autoridad Certificadora.

Para disponer de ayuda, ejecutar el programa sin argumentos, y así aparece en la consola la ayuda con las opciones para los argumentos. Si no se proporcionan los argumentos suficientes se abre el asistente de exportación de certificados. **Ejecutar pvk2pfx pasándole solo 2 argumentos:** el fichero con la clave privada (.pvk) y el fichero con el certificado correspondiente (.cer).

Al crear los certificados .pfx se muestra un cuadro de diálogo que permite optar por exportar la clave privada o no exportarla. Seleccionar que se desea exportarla. El siguiente cuadro de diálogo permite elegir tres opciones para el archivo final .PFX.

Si no seleccionamos opciones tendremos el certificado más simple posible. Denominarlo zmSERas_Simple.pfx por ejemplo.

Si seleccionamos las opciones primera y tercera se incluye en el archivo todos los certificados que permiten validar el certificado del servidor. Denominarlo zmSERas_Completo.pfx.

Al utilizar el certificado "simple" para configurar un servidor se pueden obtener mensajes de aviso indicando que falta información. Con el "completo" no deberían aparecer avisos, y por ello se recomienda usar el completo.

Al optar por exportar la clave privada al archivo PFX será necesario proporcionar una contraseña para proteger el acceso al fichero PFX. Se recomienda usar: para el servidor **conserpfx** y para el cliente **conclipfx**.

Usando las contraseñas indicadas en el guion de esta práctica siempre existe la posibilidad de recordarlas consultando nuevamente el guion de la práctica.

Al hacer doble clic sobre un fichero (.pfx) NO se abre el visor de certificados, sino que se abre el asistente de importación de certificados, ya que este formato está orientado a la transferencia de información de un computador a otro.